

FAST-FORWARD FALCON (SPEED)

- Dynamic symbol of offensive speed & evasion
- Combines aggressive systems performance with agile networking

n for Durin

finitive guide to the Zen programming lan

Zen Language Project

Zen 3.0 · ISBN 978-0-00000-000-0

Contents

1 Zen for Dummies: Complete Edition **5**
- 1.1 Table of Contents . . . 5
- 1.2 About This Book . . . 7

2 Chapter 2: Data Types **8**
- 2.1 Overview . . . 8
- 2.2 Primitive Types . . . 8
- 2.3 Collection Types . . . 9
- 2.4 Special Types . . . 10
- 2.5 Type Checking and Conversion . . . 11
- 2.6 Truthiness . . . 12
- 2.7 Type Coercion . . . 12
- 2.8 Checking for None/Null . . . 12
- 2.9 Working with Types Practically . . . 13
- 2.10 Chapter Summary . . . 13

3 Chapter 3: Operators and Expressions **14**
- 3.1 3.1 Arithmetic . . . 14
- 3.2 3.2 Comparison . . . 14
- 3.3 3.3 Logic . . . 14
- 3.4 3.4 String concatenation: + . . . 15
- 3.5 3.5 Safe member access: ?. (optional chaining) . . . 15
- 3.6 3.6 Nullish coalescing: ?? . . . 15
- 3.7 3.7 Precedence (high to low) . . . 15
- 3.8 Quick reference . . . 16

4 Chapter 5: Functions **17**
- 4.1 Defining Functions . . . 17
- 4.2 Parameters and Arguments . . . 17
- 4.3 Return Values . . . 18
- 4.4 Function Scope . . . 19
- 4.5 Recursion . . . 20
- 4.6 Higher-Order Functions . . . 20
- 4.7 Practical Examples . . . 21
- 4.8 Function Documentation . . . 22
- 4.9 Best Practices . . . 23
- 4.10 Chapter Summary . . . 23

5 Chapter 6: Lambdas and Closures **24**
- 5.1 Lambda Functions . . . 24
- 5.2 Using Lambdas as Arguments . . . 24
- 5.3 Higher-Order Functions with Lambdas . . . 25
- 5.4 Closures . . . 26
- 5.5 Practical Applications . . . 27
- 5.6 Lambda Short Syntax (If Available) . . . 29
- 5.7 When to Use Lambdas vs Functions . . . 30

5.8 Common Pitfalls . . . 30
5.9 Chapter Summary . . . 30

6 Chapter 11: Error Handling 31
6.1 11.1 Catching errors: `try` / `catch` . . . 31
6.2 11.2 Reading the real failure: `catch as e` . . . 31
6.3 11.3 Error fields: `e.msg`, `e.code`, `e.val` . . . 32
6.4 11.4 Catching specific error types . . . 32
6.5 11.5 Raising errors: `throw` . . . 32
6.6 11.6 Cleanup: `finally` . . . 33
6.7 11.7 Full example . . . 33
6.8 11.8 Common pitfalls . . . 34
6.9 Quick reference . . . 34

7 Chapter 25: crypto Module - Complete Guide 35
7.1 Importing the crypto Module . . . 35
7.2 Cryptographic Hashing . . . 35
7.3 HMAC (Hash-based Message Authentication Code) . . . 36
7.4 Cryptographic Random . . . 36
7.5 Symmetric Encryption (AES) . . . 37
7.6 Key Derivation Functions (KDF) . . . 37
7.7 Asymmetric Encryption (RSA) . . . 38
7.8 JSON Web Tokens (JWT) . . . 39
7.9 X.509 Certificates . . . 40
7.10 Base64 Encoding/Decoding . . . 41
7.11 One-Time Passwords (OTP) . . . 41
7.12 Practical Examples . . . 42
7.13 Security Best Practices . . . 44
7.14 Chapter Summary . . . 44

8 Chapter 26: Networking Fundamentals 45
8.1 What is Networking? . . . 45
8.2 The OSI Model (Simplified) . . . 45
8.3 Key Networking Concepts . . . 45
8.4 Common Protocols . . . 46
8.5 Client-Server Model . . . 46
8.6 URLs (Uniform Resource Locators) . . . 46
8.7 Zen Networking Modules . . . 47
8.8 Network Programming Workflow . . . 47
8.9 Security Considerations . . . 47
8.10 Chapter Summary . . . 47

9 Chapter 27: socket Module 49
9.1 Importing the socket Module . . . 49
9.2 Socket Types . . . 49
9.3 Creating a Socket . . . 49
9.4 TCP Client Example . . . 49
9.5 TCP Server Example . . . 50

9.6 UDP Example . . . 51
9.7 Socket Options . . . 51
9.8 Socket Methods Reference . . . 52
9.9 Error Handling . . . 52
9.10 Practical Example: Simple Chat Server . . . 52
9.11 Best Practices . . . 53
9.12 When to Use Low-Level Sockets . . . 53
9.13 Chapter Summary . . . 54

10 Chapter 28: HTTP Protocol **55**
10.1 What is HTTP? . . . 55
10.2 HTTP Versions . . . 55
10.3 HTTP Methods . . . 55
10.4 HTTP Status Codes . . . 56
10.5 HTTP Headers . . . 57
10.6 Making HTTP Requests with Zen's `requests` Module . . . 57
10.7 Building an HTTP Server . . . 59
10.8 RESTful API Design . . . 60
10.9 HTTP Cookies . . . 61
10.10 CORS (Cross-Origin Resource Sharing) . . . 61
10.11 Best Practices . . . 62
10.12 Chapter Summary . . . 62

11 Chapter 31: HTTPS & SSL/TLS Certificates **63**
11.1 What is HTTPS? . . . 63
11.2 HTTP vs HTTPS . . . 63
11.3 How TLS Works (Simplified) . . . 63
11.4 Making HTTPS Requests . . . 63
11.5 Understanding SSL Certificates . . . 64
11.6 Working with Certificates in Zen . . . 64
11.7 Creating Self-Signed Certificates (for Development) . . . 66
11.8 HTTPS Server with Certificates . . . 66
11.9 Mutual TLS (mTLS) . . . 67
11.10 Certificate Pinning . . . 67
11.11 Common SSL/TLS Issues . . . 68
11.12 Let's Encrypt & Free Certificates . . . 68
11.13 Security Best Practices . . . 68
11.14 HSTS Example . . . 69
11.15 Chapter Summary . . . 69

12 Chapter 35: Threading Fundamentals **70**
12.1 Understanding Concurrency vs Parallelism . . . 70
12.2 Creating Threads with `spawn()` . . . 70
12.3 Thread Lifecycle . . . 71
12.4 Multiple Threads . . . 71
12.5 Thread Safety Considerations . . . 72
12.6 Practical Example: Parallel Computation . . . 72
12.7 Best Practices for Threading . . . 72

12.8 Common Pitfalls . . . 72
12.9 Chapter Summary . . . 73

13 Chapter 36: Channels **74**
13.1 What are Channels? . . . 74
13.2 Creating Channels . . . 74
13.3 Sending and Receiving . . . 74
13.4 Blocking Behavior . . . 75
13.5 Producer-Consumer Pattern . . . 75
13.6 Multiple Producers, Multiple Consumers . . . 76
13.7 Closing Channels . . . 76
13.8 Buffered vs Unbuffered Channels . . . 77
13.9 Select/Multiplexing Pattern . . . 77
13.10 Practical Example: Work Queue . . . 78
13.11 Practical Example: Timeout Pattern . . . 78
13.12 Best Practices . . . 79
13.13 Chapter Summary . . . 79

14 Chapter 42: pymport - Python Interoperability **80**
14.1 42.1 Why `pymport` (and not just `import`)? . . . 80
14.2 42.2 Syntax and Binding . . . 80
14.3 42.3 Building Zen with Python Support . . . 80
14.4 42.4 Environment Setup (venv + PYTHONPATH) . . . 81
14.5 42.5 Dependency Sync with `zen py-sync` and `requirements.txt` . . . 81
14.6 42.6 Using Python from Zen . . . 82
14.7 42.7 Working Examples (tested) . . . 84
14.8 42.8 Limits and Non-Goals (v1) . . . 85
14.9 42.9 Troubleshooting . . . 85
14.10 42.10 References . . . 85

15 Chapter 44: Web Scraping & Automation **86**
15.1 Setup . . . 86
15.2 Run the whole suite . . . 87
15.3 1) Scrape + Download with `requests` + `bs4` (pymport) . . . 87
15.4 2) Stateful Scraping + Download with `mechanicalsoup` (pymport) . . . 89
15.5 3) Browser Automation with Selenium (pymport) . . . 91
15.6 4) Low-Level Control: CDP over raw TCP + WebSocket sockets . . . 94
15.7 Tips and Gotchas . . . 97
15.8 Verified Artifacts . . . 97
15.9 Appendix: Tested Scripts . . . 98

16 Chapter 45: REST API Client **99**
16.1 Understanding REST APIs . . . 99
16.2 Basic API Client Structure . . . 99
16.3 Using the API Client . . . 100
16.4 Adding Authentication . . . 101
16.5 Error Handling and Retries . . . 101
16.6 Pagination Handling . . . 102

16.7 Rate Limiting . . . 103
16.8 Building a GitHub API Client . . . 104
16.9 Testing API Clients . . . 104
16.10 Best Practices . . . 105
16.11 Chapter Summary . . . 105

17 Chapter 46: File Processing Tools 106
17.1 Reading and Writing Different Formats . . . 106
17.2 Log File Analyzer . . . 108
17.3 Batch File Processor . . . 109
17.4 Data Transformation Pipeline . . . 110
17.5 File Watcher . . . 111
17.6 Text File Merger and Splitter . . . 112
17.7 Data Validation . . . 113
17.8 Chapter Summary . . . 114

18 Chapter 47: Network Security Tools 115
18.1 Port Scanner . . . 115
18.2 Subdomain Finder . . . 115
18.3 HTTP Header Checker . . . 116
18.4 Directory Brute Forcer . . . 117
18.5 SSL Certificate Checker . . . 117
18.6 Ethical Guidelines . . . 118
18.7 Chapter Summary . . . 118

19 Chapter 48: Capstone Projects 119
19.1 Project 1: Full-Stack Blog API . . . 119
19.2 Project 2: Real-Time Chat Application . . . 121
19.3 Project 3: Web Scraper with Data Export . . . 122
19.4 Project 4: Task Scheduler . . . 123
19.5 Final Words . . . 124
19.6 Next Steps . . . 125

1 Zen for Dummies: Complete Edition

The Definitive Guide to the Zen Programming Language

1.1 Table of Contents

This comprehensive guide is organized into detailed sections covering every aspect of Zen.

1.1.1 Part 1: Fundamentals

1. Getting Started - Installation, first program, REPL
2. Data Types - Primitives, collections, type system
3. Operators and Expressions - Arithmetic, logical, bitwise

4. Control Flow - Conditionals, loops, branching
5. Functions - Defining, calling, parameters, returns
6. Lambdas and Closures - Anonymous functions, higher-order functions

1.1.2 Part 2: Core Concepts

7. Strings Deep Dive - Manipulation, formatting, methods
8. Lists and Arrays - Operations, methods, iteration
9. Dictionaries and Maps - Key-value pairs, methods
10. Modules and Imports - Creating, importing, module system
11. Error Handling - Try/catch, throw, debugging

1.1.3 Part 3: Object-Oriented & Advanced Programming

12. Classes and Objects - OOP, inheritance, polymorphism
13. File I/O - Reading, writing, `with` statement
14. Memory and References - Values, references, ownership

1.1.4 Part 4: Standard Library Modules (Detailed)

15. math Module - Mathematical functions and constants
16. random Module - Random numbers and selection
17. string Module - String utilities
18. collections Module - Advanced data structures
19. json Module - JSON parsing and serialization
20. datetime/time Module - Time and date operations
21. pathlib & os Module - File system operations
22. re (Regex) Module - Pattern matching
23. base64 & binascii - Encoding/decoding
24. hashlib Module - Hashing algorithms
25. crypto Module - Complete Guide - Cryptography, encryption, JWT, X.509

1.1.5 Part 5: Networking & Protocols

26. Networking Fundamentals - Sockets, TCP/IP basics
27. socket Module - Low-level socket programming
28. HTTP Protocol - HTTP/1.1, requests, headers
29. requests Module - High-level HTTP client
30. http Server Module - Building HTTP servers
31. HTTPS & SSL/TLS Certificates - Secure connections, certificates
32. DNS Module - Domain name resolution
33. url Module - URL parsing and manipulation
34. Raw Sockets - Packet-level networking

1.1.6 Part 6: Concurrency & Threading

35. Threading Fundamentals - Threads, spawning, execution
36. Channels - Inter-thread communication

37. Mutexes & Locks - Synchronization primitives
38. Thread Safety & Patterns - Best practices

1.1.7 Part 7: FFI, System & Interoperability

39. FFI Fundamentals - Foreign Function Interface basics
40. FFI in Practice - Calling C functions, handles
41. sys Module - System information and operations
42. pymport - Python Interoperability - Using Python from Zen (setup, sync, examples)

1.1.8 Part 8: Practical Projects

43. CLI Tools - Building command-line applications
44. Web Scraping & Automation - Bots, scrapers, automation
45. REST API Client - API integration projects
46. File Processing Tools - Data processing pipelines
47. Network Security Tools - Ethical hacking utilities
48. Capstone Projects - Full applications

1.1.9 Appendices

- Appendix A: Built-in Functions
- Appendix B: Error Codes Reference
- Appendix C: Zen CLI Reference
- Appendix D: Standard Library Index
- Appendix E: Quick Reference Card

1.2 About This Book

This book is written for complete beginners but goes deep into every aspect of Zen. Each module gets its own detailed chapter with theory, examples, common pitfalls, and exercises.

2 Chapter 2: Data Types

Understanding data types is fundamental to programming in Zen. This chapter covers all built-in types in detail.

2.1 Overview

Zen is dynamically typed - you don't need to declare types explicitly. The interpreter determines types at runtime. However, understanding types is crucial for writing correct programs.

2.2 Primitive Types

2.2.1 1. Integers (`int`)

Integers are whole numbers, positive or negative, with no decimal point.

```
let x = 42
let y = -17
let z = 0
let big = 9223372036854775807  # Large integers supported

print(type(x))  # "int"
```

Operations:

```
let a = 10
let b = 3
print(a + b)   # Addition: 13
print(a - b)   # Subtraction: 7
print(a * b)   # Multiplication: 30
print(a / b)   # Division: 3 (integer division in some contexts)
print(a % b)   # Modulo: 1
print(a ** b)  # Exponentiation: 1000 (if supported)
```

2.2.2 2. Floats (`float`)

Floating-point numbers for decimals.

```
let pi = 3.14159
let temp = -2.5
let rate = 0.05

print(type(pi))  # "float"
```

Mixing ints and floats:

```
let result = 5 + 2.5  # Becomes float
print(type(result))  # "float"
print(result)        # 7.5
```

2.2.3 3. Booleans (`bool`)

True or false values.

```
let is_active = true
let is_done = false

print(type(is_active))  # "bool"

# From comparisons
let x = 5 > 3  # true
let y = 10 == 5  # false
```

2.2.4 4. Null (`null`)

Represents absence of value or undefined state.

```
let nothing = null
print(type(nothing))  # "null"
print(nothing)        # null

# Common use cases
func find_item(id) {
    if id not_found {
        return null
    }
    return item
}
```

2.2.5 5. Strings (`string`)

Text data enclosed in double quotes.

```
let name = "Alice"
let greeting = "Hello, World!"
let empty = ""
let with_spaces = "  spaces  "

print(type(name))  # "string"
```

See Chapter 7 for detailed string operations.

2.3 Collection Types

2.3.1 6. Lists (`list`)

Ordered, mutable collections of values. Can contain mixed types.

```
# Empty list
let empty = []
```

```
# List with values
let numbers = [1, 2, 3, 4, 5]
let mixed = [1, "hello", true, null, 3.14]
let nested = [[1, 2], [3, 4]]

print(type(numbers))  # "list"
```

Characteristics: - Ordered (index starts at 0) - Mutable (can add/remove/change elements) - Heterogeneous (mixed types allowed) - Can contain any type including lists, dicts, objects

2.3.2 7. Dictionaries (`dict`)

Unordered collections of key-value pairs (or ordered in modern implementations).

```
# Empty dict
let empty = {}

# Dict with string keys
let person = {
    "name": "Bob",
    "age": 30,
    "city": "NYC"
}

# Dict with int keys
let lookup = {1: "one", 2: "two", 3: "three"}

print(type(person))  # "dict"
```

Characteristics: - Key-value pairs - Keys must be hashable (strings, ints, etc.) - Values can be any type - Mutable - Access by key

2.3.3 8. Bytes (`bytes`)

Binary data.

```
# Bytes literals (if supported)
let data = b"hello"
print(type(data))  # "bytes"
```

2.4 Special Types

2.4.1 Functions as First-Class Values

```
func add(a, b) {
    return a + b
}
```

```
let f = add  # Assign function to variable
print(f(3, 4))  # Call through variable

let op = fn(x) { return x * 2 }  # Lambda
print(op(5))
```

2.4.2 Objects (Instances of Classes)

```
class Person {}
let p = Person()
print(type(p))  # "object" or class name
```

2.5 Type Checking and Conversion

2.5.1 Checking Types

```
let x = 42
print(type(x))  # Returns type as string

if type(x) == "int" {
    print("It's an integer")
}
```

2.5.2 Type Conversions

```
# String to int
let s = "123"
let n = int(s)
print(n + 7)  # 130

# Int to string
let num = 456
let str_num = str(num)
print("Value: " + str_num)

# String to float
let f_str = "3.14"
let f = float(f_str)
print(f * 2)

# Float to int (truncates)
let f2 = 3.9
let i2 = int(f2)
print(i2)  # 3
```

```
# Bool conversions
let b1 = bool(1)      # true (non-zero/non-null is truthy)
let b2 = bool(0)      # false
let b3 = bool("")     # false (empty string is falsy)
let b4 = bool("hi")   # true
let b5 = bool([])     # false
let b6 = bool(null)   # false
```

2.6 Truthiness

Zen follows truthy/falsy rules:

Falsy values: - `false` - `null` - `0` (integer zero) - `0.0` (float zero) - `""` (empty string) - `[]` (empty list) - `{}` (empty dict)

Everything else is truthy: - Non-zero numbers - Non-empty strings - Non-empty collections - Objects - Functions

```
if [] {
    print("Won't print")
} else {
    print("Empty list is falsy")
}

if [1, 2] {
    print("Non-empty list is truthy")
}
```

2.7 Type Coercion

Be aware of how Zen handles mixed types:

```
# String concatenation
print("Age: " + 30)  # May need explicit str() conversion

# Better to be explicit
print("Age: " + str(30))
```

2.8 Checking for None/Null

```
let value = null

if value == null {
    print("Value is null")
}

if value is null {  # Alternative syntax if supported
```

```
    print("Value is null")
}
```

2.9 Working with Types Practically

```
func describe(value) {
    let t = type(value)
    if t == "int" {
        print(str(value) + " is an integer")
    } elif t == "string" {
        print("'" + value + "' is a string of length " + len(value))
    } elif t == "list" {
        print("A list with " + len(value) + " elements")
    } elif t == "dict" {
        print("A dictionary")
    } else {
        print(str(value) + " is of type " + t)
    }
}

describe(42)
describe("hello")
describe([1, 2, 3])
describe({"a": 1})
```

2.10 Chapter Summary

- **Primitives:** int, float, bool, null, string
- **Collections:** list (ordered), dict (key-value), bytes (binary)
- **Special:** functions, objects
- Use `type()` to check types
- Use `int()`, `float()`, `str()`, `bool()` for conversions
- Understand truthy/falsy for conditionals
- Zen is dynamically typed but type-aware

Next: Chapter 3 - Operators and Expressions

3 Chapter 3: Operators and Expressions

Zen's operators cover arithmetic, comparison, logic, string handling, and safe member access. This chapter lists every operator, its precedence, and common gotchas.

3.1 3.1 Arithmetic

Operator	Meaning	Example	Result
`+`	add / concat	`1 + 2`, `"a" + "b"`	`3`, `"ab"`
`-`	subtract	`5 - 3`	`2`
`*`	multiply / repeat	`3 * 2`, `"x" * 3`	`6`, `"xxx"`
`/`	divide (float)	`7 / 2`	`3.5`
`%`	modulo	`7 % 3`	`1`
`**`	power	`2 ** 3`	`8`
`-`	negate	`-x`	`-x`

Floor division: `int(7 / 2)` → `3` (there is no `//` operator; `//` starts a comment).

Division by zero raises `DivideByZero`, catchable with `try`/`catch`.

3.2 3.2 Comparison

Operator	Meaning
`==`	equal (numbers coerce: `1 == 1.0`)
`!=`	not equal
`<`	less than
`<=`	less than or equal
`>`	greater than
`>=`	greater than or equal

Comparing incompatible types (e.g. string vs number) raises `TypeError`.

3.3 3.3 Logic

Operator	Meaning	Notes
`and`	logical and	short-circuits
`or`	logical or	short-circuits
`not`	logical not	prefix
`!`	logical not	alias for `not`

Zen truthiness: everything is truthy except `nil` and `false`.

3.4 3.4 String concatenation: +

When either side of + is a string, the other side is stringified:

```
print("caught: " + e)       # error → its message
print("count: " + 42)       # int → "42"
print("flag: " + true)      # bool → "true"
print(nil + "end")          # nil → "nil"
```

3.5 3.5 Safe member access: ?. (optional chaining)

?. reads a field only if the object is non-nil; otherwise it short-circuits to nil instead of raising. Use it for nested data that might be missing.

```
let user = nil
print(user?.name)           # → nil  (no error)

let user = { name: "Zen" }
print(user?.name)           # → "Zen"

# nested: stops at the first nil
let data = nil
print(data?.a?.b?.c)        # → nil
```

Compare with strict access, which raises on nil:

```
let user = nil
print(user.name)   # TypeError: cannot read field on nil
```

3.6 3.6 Nullish coalescing: ??

?? returns its left side if it's non-nil, otherwise the right side. It's a nil-default, not a falsy-default — 0, false, and "" are all returned as-is.

```
print(nil ?? "default")     # → "default"
print("set" ?? "default")   # → "set"
print(0 ?? "default")       # → 0      (0 is not nil)
print(false ?? "default")   # → false
print(""  ?? "default")     # → ""     (empty string is not nil)
```

Chain multiple defaults:

```
print(nil ?? nil ?? "fallback")   # → "fallback"
```

Common use: reading config with a fallback:

```
let timeout = config?.timeout ?? 30
```

3.7 3.7 Precedence (high to low)

1. () grouping, [] indexing, . / ?. field access

2. `**` power
3. `-` unary negate, `not` / `!`
4. `*`, `/`, `//`, `%`
5. `+`, `-`
6. `==`, `!=`, `<`, `<=`, `>`, `>=`
7. `and`
8. `or`
9. `??` (nullish coalescing)
10. `if ... else` / `when` (conditional expressions)

Use parentheses whenever precedence is unclear.

3.8 Quick reference

```
a?.b.c          # safe field access: nil if a is nil
x ?? default    # nil default: x if non-nil, else default
"n: " + 5       # string concat with stringification
try { ... } catch as e { print(e.msg) }   # error handling (Chapter 11)
```

4 Chapter 5: Functions

Functions are reusable blocks of code that perform specific tasks. This chapter covers function definition, parameters, returns, scope, and advanced patterns.

4.1 Defining Functions

Use the `func` keyword:

```
func say_hello() {
    print("Hello!")
}

say_hello()  # Call the function
```

4.2 Parameters and Arguments

4.2.1 Basic Parameters

```
func greet(name) {
    print("Hello, " + name + "!")
}

greet("Alice")
greet("Bob")
```

4.2.2 Multiple Parameters

```
func add(a, b) {
    return a + b
}

print(add(5, 3))  # 8
```

4.2.3 Default Parameters

Provide default values for optional parameters:

```
func greet(name, greeting = "Hello") {
    print(greeting + ", " + name + "!")
}

greet("Alice")              # Hello, Alice!
greet("Bob", "Hi")         # Hi, Bob!
greet("Charlie", "Hey")    # Hey, Charlie!
```

4.2.4 Multiple Defaults

```
func create_user(name, age = 18, active = true) {
    return {
        "name": name,
        "age": age,
        "active": active
    }
}

print(create_user("Alice"))
print(create_user("Bob", 25))
print(create_user("Charlie", 30, false))
```

4.3 Return Values

4.3.1 Single Return Value

```
func square(x) {
    return x * x
}

let result = square(7)
print(result)  # 49
```

4.3.2 Early Return

```
func is_adult(age) {
    if age < 0 {
        return false
    }
    if age >= 18 {
        return true
    }
    return false
}

print(is_adult(20))   # true
print(is_adult(15))   # false
print(is_adult(-5))   # false
```

4.3.3 Returning Multiple Values

Return a list and destructure:

```
func get_coords() {
    return [10, 20]
}

let [x, y] = get_coords()
print(x, y)  # 10 20

# Or access directly
let coords = get_coords()
print(coords[0], coords[1])
```

Return a dict:

```
func divide(a, b) {
    if b == 0 {
        return {"error": "Division by zero", "ok": false}
    }
    return {"result": a / b, "ok": true}
}

let res = divide(10, 2)
if res.ok {
    print(res.result)
}
```

4.4 Function Scope

4.4.1 Local Variables

```
func test() {
    let local_var = 100
    print(local_var)  # Accessible here
}

test()
# print(local_var)  # Error: not defined
```

4.4.2 Accessing Outer Scope

```
let global = "I'm global"

func read_global() {
    print(global)  # Can read
}

read_global()
```

4.4.3 Shadowing

```
let x = 10

func test() {
    let x = 20  # Shadows outer x
    print(x)    # 20
}

test()
print(x)  # 10
```

4.5 Recursion

Functions can call themselves:

```
func factorial(n) {
    if n <= 1 {
        return 1
    }
    return n * factorial(n - 1)
}

print(factorial(5))  # 120
print(factorial(6))  # 720
```

4.5.1 Fibonacci

```
func fib(n) {
    if n <= 1 {
        return n
    }
    return fib(n - 1) + fib(n - 2)
}

print(fib(0))  # 0
print(fib(1))  # 1
print(fib(7))  # 13
print(fib(10)) # 55
```

Note: Deep recursion may hit stack limits. For deep recursion, consider iterative approaches.

4.6 Higher-Order Functions

Functions can take other functions as arguments:

```
func apply(func, value) {
    return func(value)
}

func double(x) {
    return x * 2
}

func triple(x) {
    return x * 3
}

print(apply(double, 5))  # 10
print(apply(triple, 5))  # 15
```

Functions can return functions:

```
func make_multiplier(factor) {
    func multiplier(x) {
        return x * factor
    }
    return multiplier
}

let times3 = make_multiplier(3)
let times5 = make_multiplier(5)

print(times3(10))  # 30
print(times5(10))  # 50
```

4.7 Practical Examples

4.7.1 Validation Function

```
func validate_email(email) {
    if !email.contains("@") {
        return false
    }
    if !email.contains(".") {
        return false
    }
    return true
}

print(validate_email("test@example.com"))  # true
print(validate_email("invalid"))           # false
```

4.7.2 Data Processing

```
func filter_even(numbers) {
    let result = []
    for n in numbers {
        if n % 2 == 0 {
            result.push(n)
        }
    }
    return result
}

func sum_list(numbers) {
    let total = 0
    for n in numbers {
        total = total + n
    }
    return total
}

let nums = [1, 2, 3, 4, 5, 6]
let evens = filter_even(nums)
print("Evens:", evens)
print("Sum:", sum_list(evens))
```

4.7.3 Calculator with Operations

```
func calculate(a, b, op) {
    if op == "+") { return a + b }
    elif op == "-" { return a - b }
    elif op == "*" { return a * b }
    elif op == "/" {
        if b == 0 { throw "Division by zero" }
        return a / b
    }
    else { throw "Unknown operation" }
}

print(calculate(10, 5, "+"))  # 15
print(calculate(10, 5, "*"))  # 50
```

4.8 Function Documentation

While Zen may not have formal docstrings, use comments:

```
# Calculates the area of a rectangle
# width: The width in units
```

```
# height: The height in units
# Returns: Area as float/int
func rectangle_area(width, height) {
    return width * height
}
```

4.9 Best Practices

1. **Single Responsibility** - Each function should do one thing
2. **Descriptive Names** - Use verbs: `calculate_total()`, `get_user()`
3. **Keep it Short** - If a function is too long, break it down
4. **Avoid Side Effects** - When possible, return values instead of modifying globals
5. **Use Default Parameters** - For optional functionality
6. **Early Returns** - Handle edge cases early

4.10 Chapter Summary

- Define with `func name(params) { body }`
- Call with `name(args)`
- Use `return` to return values
- Support default parameters
- Functions have local scope
- Support recursion and higher-order functions
- Functions are first-class values

Next: Chapter 6 - Lambdas and Closures

5 Chapter 6: Lambdas and Closures

Lambdas (anonymous functions) and closures are powerful features for writing concise, flexible code.

5.1 Lambda Functions

Lambdas are functions without names, defined inline with `fn`.

5.1.1 Basic Lambda

```
let double = fn(x) { return x * 2 }
print(double(5))  # 10

let square = fn(x) { return x * x }
print(square(7))  # 49
```

5.1.2 Multiple Parameters

```
let add = fn(a, b) { return a + b }
let multiply = fn(a, b) { return a * b }

print(add(3, 4))       # 7
print(multiply(3, 4))  # 12
```

5.1.3 No Parameters

```
let greet = fn() { return "Hello!" }
print(greet())
```

5.1.4 Lambdas with Blocks

```
let process = fn(x) {
    let result = x * 2 + 10
    return result
}

print(process(5))  # 20
```

5.2 Using Lambdas as Arguments

Lambdas are perfect for passing to functions:

```
func apply_twice(func, value) {
    return func(func(value))
```

```
}

print(apply_twice(fn(x) { return x + 1 }, 5))  # 7
print(apply_twice(fn(x) { return x * 2 }, 3))  # 12
```

5.3 Higher-Order Functions with Lambdas

5.3.1 Map-like Pattern

```
func map_list(list, func) {
    let result = []
    for item in list {
        result.push(func(item))
    }
    return result
}

let numbers = [1, 2, 3, 4, 5]
let squared = map_list(numbers, fn(x) { return x * x })
let doubled = map_list(numbers, fn(x) { return x * 2 })

print(squared)  # [1, 4, 9, 16, 25]
print(doubled)  # [2, 4, 6, 8, 10]
```

5.3.2 Filter-like Pattern

```
func filter_list(list, predicate) {
    let result = []
    for item in list {
        if predicate(item) {
            result.push(item)
        }
    }
    return result
}

let numbers = [1, 2, 3, 4, 5, 6, 7, 8]
let evens = filter_list(numbers, fn(x) { return x % 2 == 0 })
let odds = filter_list(numbers, fn(x) { return x % 2 == 1 })

print(evens)  # [2, 4, 6, 8]
print(odds)   # [1, 3, 5, 7]
```

5.3.3 Reduce-like Pattern

```
func reduce_list(list, func, initial) {
    let acc = initial
    for item in list {
        acc = func(acc, item)
    }
    return acc
}

let numbers = [1, 2, 3, 4, 5]
let sum = reduce_list(numbers, fn(a, b) { return a + b }, 0)
let product = reduce_list(numbers, fn(a, b) { return a * b }, 1)

print(sum)      # 15
print(product)  # 120
```

5.4 Closures

A closure is a function that captures variables from its surrounding scope.

5.4.1 Basic Closure

```
func make_counter() {
    let count = 0
    func increment() {
        count = count + 1
        return count
    }
    return increment
}

let counter = make_counter()
print(counter())  # 1
print(counter())  # 2
print(counter())  # 3
```

Each call to `make_counter()` creates a new closure with its own `count`:

```
let counter1 = make_counter()
let counter2 = make_counter()

print(counter1())  # 1
print(counter1())  # 2
print(counter2())  # 1 (separate counter)
print(counter1())  # 3
```

5.4.2 Closure with Parameters

```
func make_adder(x) {
    func adder(y) {
        return x + y
    }
    return adder
}

let add5 = make_adder(5)
let add10 = make_adder(10)

print(add5(3))   # 8
print(add10(3))  # 13
print(add5(20))  # 25
```

5.4.3 Multiple Captured Variables

```
func make_calculator(a, b) {
    func add() { return a + b }
    func subtract() { return a - b }
    func multiply() { return a * b }
    func divide() { return a / b }

    return {
        "add": add,
        "subtract": subtract,
        "multiply": multiply,
        "divide": divide
    }
}

let calc = make_calculator(10, 2)
print(calc.add())       # 12
print(calc.subtract())  # 8
print(calc.multiply())  # 20
print(calc.divide())    # 5
```

5.5 Practical Applications

5.5.1 Event Handlers Pattern

```
func create_button(label) {
    func on_click() {
        print("Button '" + label + "' clicked!")
    }
```

```
    return on_click
}

let save_btn = create_button("Save")
let cancel_btn = create_button("Cancel")

save_btn()    # Button 'Save' clicked!
cancel_btn()  # Button 'Cancel' clicked!
```

5.5.2 Private Variables

Closures can simulate private state:

```
func bank_account(initial_balance) {
    let balance = initial_balance

    func deposit(amount) {
        if amount > 0 {
            balance = balance + amount
            return balance
        }
        return balance
    }

    func withdraw(amount) {
        if amount > 0 && amount <= balance {
            balance = balance - amount
            return balance
        }
        return balance
    }

    func get_balance() {
        return balance
    }

    return {
        "deposit": deposit,
        "withdraw": withdraw,
        "get_balance": get_balance
    }
}

let account = bank_account(100)
print(account.get_balance())  # 100
account.deposit(50)
print(account.get_balance())  # 150
```

```
account.withdraw(30)
print(account.get_balance())  # 120
# balance is not directly accessible from outside
```

5.5.3 Memoization with Closures

```
func memoize(func) {
    let cache = {}

    func memoized(x) {
        let key = str(x)
        if cache.contains(key) {
            print("Cache hit for " + key)
            return cache[key]
        }
        print("Computing for " + key)
        let result = func(x)
        cache[key] = result
        return result
    }

    return memoized
}

func expensive(x) {
    # Simulate expensive computation
    return x * x
}

let fast_expensive = memoize(expensive)
print(fast_expensive(5))  # Computing for 5, returns 25
print(fast_expensive(5))  # Cache hit for 5, returns 25
print(fast_expensive(6))  # Computing for 6, returns 36
```

5.6 Lambda Short Syntax (If Available)

Some implementations may support shorter lambda syntax, but the `fn` form is standard in Zen:

```
# Standard form
let double = fn(x) { return x * 2 }

# If arrow syntax supported (check Zen version)
# let double = x => x * 2
```

5.7 When to Use Lambdas vs Functions

Use named functions when: - Function is complex or long - Function will be reused in multiple places - You want clear documentation - Function needs a descriptive name

Use lambdas when: - Function is short and simple - Used as argument to higher-order function - One-time use - Want to keep code concise

5.8 Common Pitfalls

5.8.1 Variable Capture in Loops

Be careful capturing loop variables:

```
let funcs = []
let i = 0
while i < 3 {
    let j = i  # Capture in new scope
    funcs.push(fn() { return j })
    i = i + 1
}

print(funcs[0]())  # 0
print(funcs[1]())  # 1
print(funcs[2]())  # 2
```

5.9 Chapter Summary

- Lambdas are anonymous functions created with `fn(params) { body }`
- Can be assigned to variables and passed as arguments
- Closures capture variables from their enclosing scope
- Useful for higher-order functions, callbacks, and private state
- Each closure maintains its own captured state
- Great for functional programming patterns

Next: Part 2 - Core Concepts (Strings, Lists, Dictionaries)

6 Chapter 11: Error Handling

Zen is a “safe by default” language: instead of crashing, runtime problems become **catchable errors**. This chapter covers `try`/`catch`/`throw`/`finally`, how to read the real failure out of a caught error, and the error types Zen raises.

6.1 11.1 Catching errors: `try` / `catch`

Wrap risky code in `try`. If anything inside raises an error, execution jumps to the matching `catch` block.

```
try {
    let x = 1 / 0        # raises DivideByZero
    print("never reached")
} catch {
    print("division failed")
}
# prints: division failed
```

A bare `catch` handles **any** error. If the `try` body completes normally, `catch` is skipped:

```
var ran = false
try {
    print("ok")
    ran = true
} catch {
    ran = false
}
print(ran)   # → true
```

6.2 11.2 Reading the real failure: `catch as e`

Without a binding, you only know *that* something failed, not *what*. Bind the error with `catch as e` to inspect it:

```
try {
    let y = 1 / 0
} catch as e {
    print(str(e))   # → DivideByZero: division by zero
}
```

`str(e)` renders the real message. You can also concatenate a string with the error directly:

```
catch as e {
    print("caught: " + e)   # → caught: DivideByZero: division by zero
}
```

6.3 11.3 Error fields: `e.msg`, `e.code`, `e.val`

A caught error is an object with three readable fields:

Field	Meaning	Example
`e.msg`	Human-readable message	`"DivideByZero: division by zero"`
`e.code`	Numeric error code	`0`
`e.val`	Extra attached value (often `nil`)	`nil`

```
try {
    let a = [1][5]   # IndexError
} catch as e {
    print(e.msg)   # → IndexError: index 5 out of bounds for length 1
    print(e.code)  # → 0
    print(e.val)   # → nil
}
```

6.4 11.4 Catching specific error types

You can add typed clauses to handle only certain errors. Unmatched clauses fall through to the next one, and a clause whose type doesn't match lets the error keep propagating.

```
try {
    risky()
} catch TypeError {
    print("bad type")
} catch UserError {
    print("user raised")
} catch {
    print("everything else")
}
```

Zen's built-in error types include `TypeError`, `IndexError`, `KeyError`, `DivideByZero`, `ValueError`, `NativeError`, `ForeignAttribute`, and `UserError` (raised by `throw`).

6.5 11.5 Raising errors: `throw`

Use `throw` to raise your own errors. The value becomes `e.msg` (prefixed with `UserError:` when it's a string).

```
func require_positive(n) {
    if n < 0 {
        throw "value must be positive, got " + str(n)
    }
}
```

```
try {
    require_positive(-3)
} catch as e {
    print(e.msg)   # → UserError: value must be positive, got -3
}
```

`throw` accepts any value, not just strings. The value is stringified into `e.msg` (prefixed with `UserError:`); `e.val` is always `nil` for `throw`.

```
throw { code: 42, detail: "retry later" }

# caught as:
#   e.msg → "UserError: <dict>"
#   e.val → nil
```

6.6 11.6 Cleanup: `finally`

A `finally` block runs whether the `try` succeeded or failed, and whether or not `catch` handled the error. It must come **after** any `catch` clauses.

```
var log = ""
try {
    throw "boom"
} catch {
    log = log + "c"
} finally {
    log = log + "f"
}
print(log)   # → cf
```

```
# finally also runs when nothing failed
var log2 = ""
try {
    log2 = "b"
} finally {
    log2 = log2 + "f"
}
print(log2)   # → bf
```

6.7 11.7 Full example

```
func safe_divide(a, b) {
    try {
        return a / b
    } catch as e {
        print("safe_divide failed: " + e.msg)
        return nil
```

```
    } finally {
        print("safe_divide cleanup")
    }
}

print(safe_divide(10, 2))  # → 5  (then "safe_divide cleanup")
print(safe_divide(10, 0))  # → nil (after printing the error + cleanup)
```

6.8 11.8 Common pitfalls

- **`e` only exists inside the `catch` block.** Don't reference it outside.
- **`finally` goes after `catch`, not between `try` and `catch`.**
- **A bare `catch` swallows everything.** Use typed clauses when you want to let unexpected errors propagate.
- **`str(e)` is the readable message.** Never rely on `typeof(e)` (it's just `"error"`) — use `e.msg` instead.

6.9 Quick reference

```
try {
    # risky code
} catch as e {
    print(str(e))   # real message
    print(e.msg)    # same as str(e)
    print(e.code)   # numeric code
    print(e.val)    # attached value
} catch TypeError {
    # only type errors
} finally {
    # always runs
}

throw "something went wrong"   # raise your own error
```

7 Chapter 25: crypto Module - Complete Guide

The `crypto` module provides comprehensive cryptographic functionality including hashing, encryption, decryption, digital signatures, JWT, certificates, and more.

7.1 Importing the crypto Module

```
import crypto
```

The crypto module is organized into submodules: - `crypto.hash` - Hashing functions - `crypto.hmac` - HMAC - `crypto.symmetric` - Symmetric encryption (AES, etc.) - `crypto.asymmetric` - Asymmetric encryption (RSA, etc.) - `crypto.kdf` - Key derivation functions - `crypto.rand` - Cryptographic random - `crypto.jwt` - JSON Web Tokens - `crypto.x509` - X.509 certificates - `crypto.encoding` - Encoding utilities - `crypto.otp` - One-time passwords

7.2 Cryptographic Hashing

7.2.1 Hash Functions

```
import crypto.hash

# SHA-256
let h = crypto.hash.sha256("Hello World")
print(h.hexdigest())

# SHA-512
let h2 = crypto.hash.sha512("Hello World")
print(h2.hexdigest())

# SHA-1 (less secure, avoid for new applications)
let h3 = crypto.hash.sha1("Hello World")
print(h3.hexdigest())

# MD5 (not cryptographically secure)
let h4 = crypto.hash.md5("Hello World")
print(h4.hexdigest())

# Blake2b
let h5 = crypto.hash.blake2b("Hello World")
print(h5.hexdigest())
```

7.2.2 Hashing Files

```
import crypto.hash
```

```
func hash_file(filename, algorithm) {
    let content = fs.read(filename)
    if algorithm == "sha256" {
        return crypto.hash.sha256(content).hexdigest()
    }
    return null
}

print(hash_file("important.txt", "sha256"))
```

7.3 HMAC (Hash-based Message Authentication Code)

```
import crypto.hmac

let key = "secret-key"
let message = "Hello World"

let mac = crypto.hmac.new(key, message, "sha256")
print(mac.hexdigest())
```

7.3.1 Verifying HMAC

```
func verify_hmac(message, key, signature) {
    let mac = crypto.hmac.new(key, message, "sha256")
    return mac.hexdigest() == signature
}
```

7.4 Cryptographic Random

```
import crypto.rand

# Random bytes
let bytes = crypto.rand.bytes(32)
print(bytes)

# Random integer
let num = crypto.rand.int(1, 100)
print(num)

# Random string
let token = crypto.rand.token(32)
print(token)  # Hex string
```

7.5 Symmetric Encryption (AES)

Symmetric encryption uses the same key for encryption and decryption.

7.5.1 AES-GCM Encryption

```
import crypto.symmetric
import crypto.rand

# Generate key (32 bytes for AES-256)
let key = crypto.rand.bytes(32)
let plaintext = "Secret message"

# Encrypt
let encrypted = crypto.symmetric.aes_gcm_encrypt(key, plaintext)
print("Nonce:", encrypted.nonce)
print("Ciphertext:", encrypted.ciphertext)
print("Tag:", encrypted.tag)

# Decrypt
let decrypted = crypto.symmetric.aes_gcm_decrypt(key, encrypted.nonce,
    encrypted.ciphertext, encrypted.tag)
print("Decrypted:", decrypted)
```

7.5.2 AES-CBC Encryption

```
import crypto.symmetric
import crypto.rand

let key = crypto.rand.bytes(32)
let iv = crypto.rand.bytes(16)
let plaintext = "Another secret message"

let ciphertext = crypto.symmetric.aes_cbc_encrypt(key, iv, plaintext)
let decrypted = crypto.symmetric.aes_cbc_decrypt(key, iv, ciphertext)
print(decrypted)
```

7.6 Key Derivation Functions (KDF)

Used to derive encryption keys from passwords.

7.6.1 PBKDF2

```
import crypto.kdf
import crypto.rand
```

```
let password = "my_password"
let salt = crypto.rand.bytes(16)

# Derive key
let key = crypto.kdf.pbkdf2(password, salt, iterations=100000, length=32)
print(key)
```

7.6.2 Argon2 (if supported)

```
import crypto.kdf

let password = "my_password"
let salt = crypto.rand.bytes(16)
let key = crypto.kdf.argon2(password, salt, length=32)
print(key)
```

7.7 Asymmetric Encryption (RSA)

Asymmetric uses public key for encryption, private key for decryption.

7.7.1 Generating RSA Key Pair

```
import crypto.asymmetric

let keypair = crypto.asymmetric.rsa_generate(2048)
print("Public key generated")
print("Private key generated")
```

7.7.2 RSA Encryption/Decryption

```
import crypto.asymmetric

let keypair = crypto.asymmetric.rsa_generate(2048)
let plaintext = "Secret message"

# Encrypt with public key
let ciphertext = crypto.asymmetric.rsa_encrypt(keypair.public_key, plaintext)

# Decrypt with private key
let decrypted = crypto.asymmetric.rsa_decrypt(keypair.private_key, ciphertext)
print(decrypted)
```

7.7.3 RSA Signatures

```
import crypto.asymmetric
import crypto.hash

let keypair = crypto.asymmetric.rsa_generate(2048)
let message = "Important message"

# Sign
let signature = crypto.asymmetric.rsa_sign(keypair.private_key, message, "sha256")

# Verify
let valid = crypto.asymmetric.rsa_verify(keypair.public_key, message, signature, "sha25
print("Valid:", valid)
```

7.8 JSON Web Tokens (JWT)

JWTs are used for authentication and information exchange.

7.8.1 Creating a JWT

```
import crypto.jwt

let secret = "my-secret-key"
let payload = {
    "user_id": 123,
    "username": "alice",
    "exp": time.time() + 3600  # Expires in 1 hour
}

let token = crypto.jwt.encode(payload, secret, algorithm="HS256")
print(token)
```

7.8.2 Verifying and Decoding a JWT

```
import crypto.jwt

let token = "eyJhbGc..."
let secret = "my-secret-key"

try {
    let payload = crypto.jwt.decode(token, secret, algorithm="HS256")
    print("User ID:", payload.user_id)
    print("Valid token")
} catch (e) {
```

```
    print("Invalid token:", e)
}
```

7.8.3 JWT with RS256 (Asymmetric)

```
import crypto.jwt
import crypto.asymmetric

let keypair = crypto.asymmetric.rsa_generate(2048)
let payload = {"user_id": 123}

let token = crypto.jwt.encode(payload, keypair.private_key, algorithm="RS256")
let decoded = crypto.jwt.decode(token, keypair.public_key, algorithm="RS256")
```

7.9 X.509 Certificates

Working with SSL/TLS certificates.

7.9.1 Loading a Certificate

```
import crypto.x509

let cert_pem = fs.read("certificate.crt")
let cert = crypto.x509.load_cert(cert_pem)

print("Subject:", cert.subject)
print("Issuer:", cert.issuer)
print("Valid from:", cert.not_before)
print("Valid until:", cert.not_after)
```

7.9.2 Verifying Certificate

```
import crypto.x509

let cert = crypto.x509.load_cert(fs.read("cert.crt"))
let ca_cert = crypto.x509.load_cert(fs.read("ca.crt"))

let valid = crypto.x509.verify(cert, ca_cert)
print("Certificate valid:", valid)
```

7.9.3 Getting Certificate Information

```
let cert = crypto.x509.load_cert(pem)
print(cert.serial_number)
```

```
print(cert.signature_algorithm)
print(cert.public_key)
```

7.10 Base64 Encoding/Decoding

```
import crypto.encoding

let data = "Hello World"
let encoded = crypto.encoding.base64_encode(data)
print(encoded)  # SGVsbG8gV29ybGQ=

let decoded = crypto.encoding.base64_decode(encoded)
print(decoded)  # Hello World

# URL-safe base64
let url_encoded = crypto.encoding.base64url_encode(data)
let url_decoded = crypto.encoding.base64url_decode(url_encoded)
```

7.11 One-Time Passwords (OTP)

7.11.1 TOTP (Time-based OTP)

```
import crypto.otp

let secret = crypto.otp.generate_secret()
print("Secret:", secret)

# Generate TOTP code
let code = crypto.otp.totp(secret)
print("Current code:", code)

# Verify TOTP code
let valid = crypto.otp.verify_totp(secret, "123456")
print("Valid:", valid)
```

7.11.2 HOTP (HCounter-based OTP)

```
import crypto.otp

let secret = crypto.otp.generate_secret()
let counter = 0

let code = crypto.otp.hotp(secret, counter)
print(code)
counter = counter + 1
```

7.12 Practical Examples

7.12.1 Password Hashing

```
import crypto.kdf
import crypto.rand

func hash_password(password) {
    let salt = crypto.rand.bytes(16)
    let hash = crypto.kdf.pbkdf2(password, salt, iterations=100000, length=32)
    return {
        "hash": hash,
        "salt": salt
    }
}

func verify_password(password, stored) {
    let hash = crypto.kdf.pbkdf2(password, stored.salt, iterations=100000, length=32)
    return hash == stored.hash
}

let stored = hash_password("mysecretpassword")
print(verify_password("mysecretpassword", stored))  # true
print(verify_password("wrongpassword", stored))     # false
```

7.12.2 Secure File Encryption

```
import crypto.symmetric
import crypto.rand
import crypto.kdf

func encrypt_file(password, input_file, output_file) {
    let salt = crypto.rand.bytes(16)
    let key = crypto.kdf.pbkdf2(password, salt, iterations=100000, length=32)
    let plaintext = fs.read(input_file)
    let encrypted = crypto.symmetric.aes_gcm_encrypt(key, plaintext)

    # Save salt + nonce + ciphertext + tag
    let data = salt + encrypted.nonce + encrypted.ciphertext + encrypted.tag
    fs.write_bytes(output_file, data)
}

func decrypt_file(password, input_file, output_file) {
    let data = fs.read_bytes(input_file)
    let salt = data[0:16]
    let nonce = data[16:28]
    let tag = data[-16:]
```

```
    let ciphertext = data[28:-16]

    let key = crypto.kdf.pbkdf2(password, salt, iterations=100000, length=32)
    let plaintext = crypto.symmetric.aes_gcm_decrypt(key, nonce, ciphertext, tag)
    fs.write(output_file, plaintext)
}
```

7.12.3 API Authentication with JWT

```
import crypto.jwt
import http
import json

let JWT_SECRET = "super-secret-key-change-in-production"

func auth_handler(req) {
    if req.path == "/login" && req.method == "POST" {
        let body = json.parse(req.body)
        # Validate credentials (simplified)
        if body.username == "admin" && body.password == "pass" {
            let token = crypto.jwt.encode({
                "user": body.username,
                "exp": time.time() + 3600
            }, JWT_SECRET)
            return {"status": 200, "body": json.stringify({"token": token})}
        }
        return {"status": 401, "body": json.stringify({"error": "Invalid credentials"})
    }

    # Protected route
    if req.path == "/protected" {
        let auth = req.headers.get("Authorization", "")
        if auth.starts_with("Bearer ") {
            let token = auth.split(" ")[1]
            try {
                let payload = crypto.jwt.decode(token, JWT_SECRET)
                return {"status": 200, "body": json.stringify({"message": "Hello " + pa
            } catch {
                return {"status": 401, "body": json.stringify({"error": "Invalid token"
            }
        }
        return {"status": 401, "body": json.stringify({"error": "No token"})}
    }

    return {"status": 404, "body": "Not found"}
}
```

```
http.serve("0.0.0.0", 8080, auth_handler)
```

7.13 Security Best Practices

1. **Use strong algorithms** - SHA-256+, AES-256, RSA-2048+
2. **Never use MD5/SHA-1** for security
3. **Use cryptographically secure random** - Always use `crypto.rand`
4. **Derive keys from passwords** - Use PBKDF2/Argon2, not raw hashes
5. **Use authenticated encryption** - AES-GCM provides encryption + authentication
6. **Store secrets securely** - Never hardcode in production
7. **Use HTTPS** - Always encrypt in transit
8. **Verify signatures** - Always verify JWTs and signatures
9. **Keep libraries updated** - Use latest secure versions
10. **Salt passwords** - Never hash passwords without salt

7.14 Chapter Summary

- Crypto module provides comprehensive cryptographic tools
- Use hash functions for integrity, HMAC for authentication
- Use symmetric encryption (AES-GCM) for data encryption
- Use asymmetric (RSA) for key exchange and signatures
- Use KDFs (PBKDF2) for password-based keys
- JWT for token-based authentication
- Always follow cryptographic best practices
- Use secure random for keys and nonces

Next: Part 5 continues with HTTPS & SSL Certificates

8 Chapter 26: Networking Fundamentals

Before diving into Zen's networking modules, it's important to understand the basics of computer networking, protocols, and how they work.

8.1 What is Networking?

Networking allows computers to communicate with each other over a network (like the internet). In Zen, you can build clients, servers, and work with various protocols.

8.2 The OSI Model (Simplified)

Understanding layers helps when working with networking:

Layer	Name	What it does	Examples
7	Application	User-facing protocols	HTTP, HTTPS, DNS, FTP
6	Presentation	Data formatting/encryption	SSL/TLS, JSON
5	Session	Managing connections	Sessions, cookies
4	Transport	Reliable delivery	TCP, UDP
3	Network	Routing between networks	IP (IPv4/IPv6)
2	Data Link	Local network delivery	Ethernet, WiFi
1	Physical	Hardware transmission	Cables, radio

8.3 Key Networking Concepts

8.3.1 IP Addresses

Unique identifiers for devices on a network: - **IPv4**: `192.168.1.1`, `8.8.8.8` (32-bit) - **IPv6**: `2001:4860:4860::8888` (128-bit) - **localhost/loopback**: `127.0.0.1`

8.3.2 Ports

Ports identify specific services on a device (0-65535): - **Well-known ports** (0-1023): HTTP(80), HTTPS(443), SSH(22), FTP(21) - **Registered ports** (1024-49151): Application-specific - **Dynamic/private** (49152-65535): Temporary use

8.3.3 Sockets

A socket is an endpoint for communication - combination of IP address + port.

8.3.4 TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Guaranteed delivery	No guarantee
Ordering	Ordered delivery	No ordering

Feature	TCP	UDP
Speed	Slower (overhead)	Faster
Use cases	HTTP, HTTPS, FTP, Email	DNS, Streaming, Gaming

8.4 Common Protocols

8.4.1 HTTP (HyperText Transfer Protocol)

- Port 80
- Request/Response model
- Stateless
- Used for web pages and APIs

8.4.2 HTTPS (HTTP Secure)

- Port 443
- HTTP over TLS/SSL encryption
- Secure communication

8.4.3 DNS (Domain Name System)

- Port 53
- Translates domain names to IP addresses
- `example.com` → `93.184.216.34`

8.4.4 FTP (File Transfer Protocol)

- Ports 20, 21
- File transfers

8.5 Client-Server Model

Most network applications follow this model: - **Server**: Listens for connections, provides services - **Client**: Connects to server, requests services

```
# Conceptually
# Server waits for connections on port 8080
# Client connects to server:8080 and requests data
```

8.6 URLs (Uniform Resource Locators)

Structure: `scheme://host:port/path?query#fragment`

Example: `https://example.com:443/products?id=123#details` - scheme: `https` - host: `example.com` - port: `443` - path: `/products` - query: `id=123` - fragment: `details`

8.7 Zen Networking Modules

Zen provides several modules for networking:

Module	Purpose	Level
`socket`	Low-level BSD sockets	Very low
`http`	HTTP server	High-level
`requests`	HTTP client	High-level
`dns`	DNS lookups	Mid-level
`url`	URL parsing	Utility
`raw_socket`	Raw packet access	Very low

8.8 Network Programming Workflow

8.8.1 For a Client (e.g., HTTP request)

1. Resolve hostname to IP (DNS)
2. Create socket
3. Connect to server
4. Send request
5. Receive response
6. Close connection

8.8.2 For a Server

1. Create socket
2. Bind to IP and port
3. Listen for connections
4. Accept connections
5. Handle requests
6. Send responses
7. Close

8.9 Security Considerations

When doing network programming: - **Validate input** - Never trust network data - **Use HTTPS** - Encrypt sensitive data - **Check certificates** - Verify SSL/TLS certs - **Handle errors** - Network can fail - **Rate limiting** - Be respectful to servers - **Firewalls** - Consider network restrictions

8.10 Chapter Summary

- Networking enables inter-computer communication
- TCP is reliable, UDP is fast but unreliable
- Ports identify services, IPs identify hosts
- HTTP/HTTPS for web, DNS for name resolution
- Zen has socket, http, requests, dns modules

- Understand client-server model
- Security is crucial in network programming

Next: Chapter 27 - socket Module

9 Chapter 27: socket Module

The `socket` module provides low-level BSD socket API access in Zen. This gives you full control over network connections but requires understanding socket programming concepts.

9.1 Importing the socket Module

```
import socket
```

9.2 Socket Types

9.2.1 Socket Families

- `socket.AF_INET` - IPv4
- `socket.AF_INET6` - IPv6
- `socket.AF_UNIX` - Unix domain sockets (local)

9.2.2 Socket Types

- `socket.SOCK_STREAM` - TCP (reliable, connection-oriented)
- `socket.SOCK_DGRAM` - UDP (unreliable, connectionless)
- `socket.SOCK_RAW` - Raw sockets

9.3 Creating a Socket

```
# TCP socket
let tcp_sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)

# UDP socket
let udp_sock = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
```

9.4 TCP Client Example

```
import socket

# Create TCP socket
let s = socket.create(socket.AF_INET, socket.SOCK_STREAM)

# Connect to server
socket.connect(s, "example.com", 80)

# Send HTTP request
let request = "GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n"
socket.send(s, request)
```

```
# Receive response
let response = socket.recv(s, 4096)
print(response)

# Close socket
socket.close(s)
```

9.5 TCP Server Example

```
import socket

# Create socket
let server = socket.create(socket.AF_INET, socket.SOCK_STREAM)

# Set socket options to reuse address
socket.setsockopt(server, socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# Bind to address and port
socket.bind(server, "0.0.0.0", 8080)

# Listen for connections (backlog of 5)
socket.listen(server, 5)

print("Server listening on port 8080...")

while true {
    # Accept connection
    let client = socket.accept(server)
    print("Client connected")

    # Receive data
    let data = socket.recv(client, 1024)
    print("Received: " + data)

    # Send response
    let response = "HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\n\r\nHello from Zen s
    socket.send(client, response)

    # Close client connection
    socket.close(client)
}
```

9.6 UDP Example

9.6.1 UDP Client

```
import socket

let s = socket.create(socket.AF_INET, socket.SOCK_DGRAM)

let message = "Hello UDP Server"
socket.sendto(s, message, "localhost", 9999)

let [data, addr] = socket.recvfrom(s, 1024)
print("Received: " + data)
print("From: " + addr)

socket.close(s)
```

9.6.2 UDP Server

```
import socket

let s = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
socket.bind(s, "0.0.0.0", 9999)

print("UDP server listening on port 9999...")

while true {
    let [data, addr] = socket.recvfrom(s, 1024)
    print("Received from " + addr + ": " + data)

    # Echo back
    socket.sendto(s, "Echo: " + data, addr[0], addr[1])
}
```

9.7 Socket Options

```
import socket

let s = socket.create(socket.AF_INET, socket.SOCK_STREAM)

# Allow address reuse
socket.setsockopt(s, socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# Set timeout
socket.settimeout(s, 5)  # 5 seconds
```

```
# Get socket option
let val = socket.getsockopt(s, socket.SOL_SOCKET, socket.SO_REUSEADDR)
```

9.8 Socket Methods Reference

Method	Description
`create(family, type)`	Create a new socket
`bind(sock, addr, port)`	Bind socket to address/port
`listen(sock, backlog)`	Listen for connections
`accept(sock)`	Accept incoming connection
`connect(sock, addr, port)`	Connect to remote host
`send(sock, data)`	Send data on connected socket
`recv(sock, size)`	Receive data from connected socket
`sendto(sock, data, addr, port)`	Send data to address (UDP)
`recvfrom(sock, size)`	Receive data with sender address (UDP)
`close(sock)`	Close socket
`setsockopt(sock, level, opt, value)`	Set socket option
`getsockopt(sock, level, opt)`	Get socket option
`settimeout(sock, seconds)`	Set timeout

9.9 Error Handling

```
import socket

try {
    let s = socket.create(socket.AF_INET, socket.SOCK_STREAM)
    socket.connect(s, "invalid.host.example", 80)
} catch (e) {
    print("Socket error: " + e)
    if e.contains("timeout") {
        print("Connection timed out")
    }
}
```

9.10 Practical Example: Simple Chat Server

```
import socket

let server = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.setsockopt(server, socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
socket.bind(server, "0.0.0.0", 9000)
socket.listen(server, 10)

print("Chat server on port 9000")
```

```
let clients = []

while true {
    let client = socket.accept(server)
    clients.push(client)
    print("New client connected")

    # Handle client in separate thread
    spawn(fn() {
        while true {
            try {
                let msg = socket.recv(client, 1024)
                if len(msg) == 0 {
                    break
                }
                print("Broadcasting: " + msg)
                # Broadcast to all clients
                for c in clients {
                    if c != client {
                        try {
                            socket.send(c, msg)
                        } catch {}
                    }
                }
            } catch {
                break
            }
        }
        socket.close(client)
    })
}
```

9.11 Best Practices

1. **Always close sockets** - Use try/finally or track closures
2. **Handle errors** - Network operations can fail
3. **Set timeouts** - Avoid hanging indefinitely
4. **Reuse addresses** - Use SO_REUSEADDR for servers
5. **Use threads** - Handle multiple clients concurrently
6. **Validate data** - Don't trust received data
7. **Consider using high-level APIs** - `requests` is easier for HTTP

9.12 When to Use Low-Level Sockets

Use the `socket` module when: - You need custom protocols - Working with non-HTTP protocols - Need fine-grained control - Building custom servers/clients - Working with UDP/TCP directly

Use higher-level modules (`requests`, `http`) when: - Working with HTTP/HTTPS - Simpler use cases - Less error-prone

9.13 Chapter Summary

- `socket` module provides BSD socket API
- TCP uses SOCK_STREAM, UDP uses SOCK_DGRAM
- Client: create → connect → send/recv → close
- Server: create → bind → listen → accept → send/recv → close
- Use `sendto`/`recvfrom` for UDP
- Handle errors and use threading for concurrent clients
- Set socket options for flexibility

Next: Chapter 28 - HTTP Protocol

10 Chapter 28: HTTP Protocol

HTTP (HyperText Transfer Protocol) is the foundation of data communication on the web. This chapter explains HTTP in detail and how to work with it in Zen.

10.1 What is HTTP?

HTTP is an application-layer protocol for transmitting hypermedia documents. It's: - **Stateless**: Each request is independent - **Client-Server**: Client requests, server responds - **Text-based**: Human-readable protocol (though can carry binary) - **Request-Response**: Follows this pattern

10.2 HTTP Versions

Version	Status	Notes
HTTP/1.0	Legacy	Simple, one request per connection
HTTP/1.1	Widely used	Persistent connections, chunked transfer
HTTP/2	Modern	Multiplexing, binary, server push
HTTP/3	New	Uses QUIC over UDP

10.3 HTTP Methods

HTTP methods (verbs) indicate the action to perform:

Method	Purpose	Idempotent	Safe	Body
GET	Retrieve resource	Yes	Yes	Usually no
POST	Create resource / submit data	No	No	Yes
PUT	Update/replace resource (full)	Yes	No	Yes
PATCH	Partial update	No	No	Yes
DELETE	Remove resource	Yes	No	Usually no
HEAD	Same as GET but no body	Yes	Yes	No
OPTIONS	Get supported methods	Yes	Yes	No

10.3.1 Examples

```
# GET - retrieve data
GET /users/123 HTTP/1.1
Host: api.example.com

# POST - create data
POST /users HTTP/1.1
Host: api.example.com
Content-Type: application/json

{"name": "Alice", "email": "alice@example.com"}
```

```
# PUT - replace entire resource
PUT /users/123 HTTP/1.1
Host: api.example.com
Content-Type: application/json

{"name": "Alice Smith", "email": "alice@example.com", "age": 30}

# DELETE - remove resource
DELETE /users/123 HTTP/1.1
Host: api.example.com
```

10.4 HTTP Status Codes

Status codes indicate the result of the request:

10.4.1 1xx Informational

- 100 Continue
- 101 Switching Protocols

10.4.2 2xx Success

- 200 OK - Request succeeded
- 201 Created - Resource created
- 204 No Content - Success but no body

10.4.3 3xx Redirection

- 301 Moved Permanently
- 302 Found (temporary redirect)
- 304 Not Modified - Use cached version

10.4.4 4xx Client Errors

- 400 Bad Request - Invalid syntax
- 401 Unauthorized - Authentication required
- 403 Forbidden - Not allowed
- 404 Not Found - Resource doesn’t exist
- 405 Method Not Allowed
- 422 Unprocessable Entity
- 429 Too Many Requests

10.4.5 5xx Server Errors

- 500 Internal Server Error
- 502 Bad Gateway
- 503 Service Unavailable

- 504 Gateway Timeout

10.5 HTTP Headers

Headers provide additional information about the request/response.

10.5.1 Common Request Headers

```
Host: example.com
User-Agent: Zen/3.0
Accept: text/html,application/json
Content-Type: application/json
Authorization: Bearer token123
Cookie: session=abc123
```

10.5.2 Common Response Headers

```
Content-Type: text/html; charset=utf-8
Content-Length: 1234
Cache-Control: max-age=3600
Set-Cookie: session=xyz789
Server: nginx
Access-Control-Allow-Origin: *
```

10.6 Making HTTP Requests with Zen's `requests` Module

The easiest way to make HTTP requests:

```
import requests

# GET request
let resp = requests.get("https://httpbin.org/get")
print("Status:", resp.status_code)
print("Body:", resp.text)
```

10.6.1 GET with Query Parameters

```
import requests

let resp = requests.get("https://httpbin.org/get",
    params={"name": "Alice", "age": "30"}
)
print(resp.text)
```

10.6.2 POST with JSON

```
import requests
import json

let data = {"username": "alice", "password": "secret"}
let resp = requests.post("https://httpbin.org/post",
    json=data,
    headers={"User-Agent": "Zen-App/1.0"}
)
print(resp.status_code)
let body = json.parse(resp.text)
print(body.json)
```

10.6.3 POST with Form Data

```
import requests

let resp = requests.post("https://httpbin.org/post",
    data={"key1": "value1", "key2": "value2"}
)
print(resp.text)
```

10.6.4 Custom Headers

```
import requests

let resp = requests.get("https://httpbin.org/headers",
    headers={
        "Authorization": "Bearer mytoken",
        "X-Custom-Header": "hello"
    }
)
print(resp.text)
```

10.6.5 Handling Response

```
import requests
import json

let resp = requests.get("https://httpbin.org/status/200")

print("Status Code:", resp.status_code)
print("Headers:", resp.headers)
print("Text:", resp.text)
```

```
print("URL:", resp.url)

# If JSON response
if resp.headers["Content-Type"].contains("json") {
    let data = resp.json()
    print(data)
}
```

10.7 Building an HTTP Server

Use the `http` module to create servers:

```
import http
import json

func handle_request(req) {
    let path = req.path
    let method = req.method

    if path == "/" && method == "GET" {
        return {
            "status": 200,
            "headers": {"Content-Type": "text/html"},
            "body": "<h1>Welcome to Zen HTTP Server</h1>"
        }
    }

    if path == "/api/data" && method == "GET" {
        let data = {"message": "Hello", "count": 42}
        return {
            "status": 200,
            "headers": {"Content-Type": "application/json"},
            "body": json.stringify(data)
        }
    }

    if path == "/api/echo" && method == "POST" {
        return {
            "status": 200,
            "headers": {"Content-Type": "application/json"},
            "body": json.stringify({"echo": req.body})
        }
    }

    return {
        "status": 404,
        "headers": {"Content-Type": "text/html"},
```

```
        "body": "<h1>404 Not Found</h1>"
    }
}

print("Server starting on http://localhost:8080")
http.serve("0.0.0.0", 8080, handle_request)
```

10.8 RESTful API Design

```
import http
import json

let users = [
    {"id": 1, "name": "Alice"},
    {"id": 2, "name": "Bob"}
]

func api_handler(req) {
    if req.method == "GET" && req.path == "/api/users" {
        return {
            "status": 200,
            "headers": {"Content-Type": "application/json"},
            "body": json.stringify(users)
        }
    }

    if req.method == "GET" && req.path.starts_with("/api/users/") {
        let id = int(req.path.split("/")[3])
        for user in users {
            if user.id == id {
                return {
                    "status": 200,
                    "headers": {"Content-Type": "application/json"},
                    "body": json.stringify(user)
                }
            }
        }
        return {"status": 404, "body": json.stringify({"error": "Not found"})}
    }

    return {"status": 404, "body": "Not Found"}
}

http.serve("0.0.0.0", 3000, api_handler)
```

10.9 HTTP Cookies

```
import http

func handler(req) {
    # Read cookies
    let cookies = req.cookies
    if cookies.contains("session") {
        return {"status": 200, "body": "Welcome back!"}
    }

    # Set cookie in response
    return {
        "status": 200,
        "headers": {
            "Set-Cookie": "session=abc123; Path=/; HttpOnly"
        },
        "body": "Cookie set!"
    }
}
```

10.10 CORS (Cross-Origin Resource Sharing)

```
func handler(req) {
    if req.method == "OPTIONS" {
        return {
            "status": 200,
            "headers": {
                "Access-Control-Allow-Origin": "*",
                "Access-Control-Allow-Methods": "GET, POST, PUT, DELETE",
                "Access-Control-Allow-Headers": "Content-Type, Authorization"
            },
            "body": ""
        }
    }

    return {
        "status": 200,
        "headers": {
            "Access-Control-Allow-Origin": "*",
            "Content-Type": "application/json"
        },
        "body": json.stringify({"data": "hello"})
    }
}
```

10.11 Best Practices

1. **Use appropriate methods** - GET for reads, POST for creates, etc.
2. **Handle errors gracefully** - Check status codes
3. **Set proper headers** - Content-Type, User-Agent
4. **Use HTTPS** - For sensitive data
5. **Validate input** - On both client and server
6. **Use timeouts** - Don't wait forever
7. **Respect rate limits** - Don't spam servers
8. **Follow REST conventions** - For APIs

10.12 Chapter Summary

- HTTP is stateless request-response protocol
- Methods: GET, POST, PUT, PATCH, DELETE
- Status codes indicate success/failure
- Headers carry metadata
- Use `requests` module for clients, `http` module for servers
- Handle JSON with `json` module
- Follow RESTful patterns for APIs

Next: Chapter 29 - requests Module

11 Chapter 31: HTTPS & SSL/TLS Certificates

HTTPS (HyperText Transfer Protocol Secure) is HTTP layered over TLS/SSL encryption. This chapter covers how to work with secure connections, certificates, and certificate validation in Zen.

11.1 What is HTTPS?

HTTPS uses TLS (Transport Layer Security) to encrypt communication between client and server. It provides:

- **Confidentiality** - Data is encrypted, can't be read by eavesdroppers
- **Integrity** - Data can't be modified in transit
- **Authentication** - Verifies the server's identity via certificates

11.2 HTTP vs HTTPS

Aspect	HTTP	HTTPS
Port	80	443
Encryption	None	TLS/SSL encrypted
Security	Vulnerable	Secure
URL Scheme	`http://`	`https://`
Certificate	Not required	Required

11.3 How TLS Works (Simplified)

1. **Client Hello** - Client sends supported TLS versions and cipher suites
2. **Server Hello** - Server responds with chosen version and certificate
3. **Certificate Verification** - Client verifies server's certificate
4. **Key Exchange** - Establish shared encryption keys
5. **Encrypted Communication** - All data encrypted with session keys

11.4 Making HTTPS Requests

The `requests` module handles HTTPS automatically:

```
import requests

# HTTPS request - certificate verification is enabled by default
let resp = requests.get("https://api.github.com/users/octocat")
print(resp.status_code)
print(resp.text)
```

11.4.1 Disabling Certificate Verification (Not Recommended)

Only do this for testing/development:

```
import requests

# WARNING: Insecure - only for testing
let resp = requests.get("https://self-signed.badssl.com/", verify=false)
print(resp.status_code)
```

11.4.2 Using Custom CA Certificates

```
import requests

let resp = requests.get("https://internal.company.com/api",
    verify="/path/to/custom-ca-cert.pem"
)
print(resp.status_code)
```

11.5 Understanding SSL Certificates

11.5.1 Certificate Components

- **Subject** - Who the certificate is issued to (domain name)
- **Issuer** - Certificate Authority (CA) that issued it
- **Valid From/To** - Validity period
- **Public Key** - Used for encryption/verification
- **Signature** - CA's digital signature
- **Serial Number** - Unique identifier

11.5.2 Certificate Types

Type	Description	Use Case
DV (Domain Validated)	Basic validation of domain ownership	Personal sites, blogs
OV (Organization Validated)	Validates organization identity	Business websites
EV (Extended Validation)	Strict validation process	Banks, e-commerce
Wildcard	Covers subdomains (*.example.com)	Multiple subdomains
Self-Signed	Signed by yourself, not trusted by browsers	Development, internal use

11.6 Working with Certificates in Zen

11.6.1 Inspecting Certificates

```
import crypto.x509
import fs

let cert_pem = fs.read("server.crt")
let cert = crypto.x509.load_cert(cert_pem)

print("Subject:", cert.subject)
print("Issuer:", cert.issuer)
print("Serial:", cert.serial_number)
print("Valid From:", cert.not_before)
print("Valid Until:", cert.not_after)
print("Signature Algorithm:", cert.signature_algorithm)
```

11.6.2 Checking Certificate Expiration

```
import crypto.x509
import time

func check_cert_expiry(cert_path) {
    let cert_pem = fs.read(cert_path)
    let cert = crypto.x509.load_cert(cert_pem)

    let now = time.time()
    let expiry = cert.not_after_timestamp

    if expiry < now {
        print("Certificate EXPIRED!")
        return false
    }

    let days_left = (expiry - now) / 86400
    print("Days until expiry: " + int(days_left))

    if days_left < 30 {
        print("WARNING: Certificate expiring soon!")
    }

    return true
}

check_cert_expiry("server.crt")
```

11.6.3 Verifying Certificate Chain

```
import crypto.x509

func verify_chain(server_cert_path, ca_cert_path) {
    let server_cert = crypto.x509.load_cert(fs.read(server_cert_path))
    let ca_cert = crypto.x509.load_cert(fs.read(ca_cert_path))

    try {
        let valid = crypto.x509.verify(server_cert, ca_cert)
        if valid {
            print("Certificate chain is valid")
        } else {
            print("Certificate chain is invalid")
        }
        return valid
    } catch (e) {
        print("Verification error: " + e)
        return false
    }
}
```

11.7 Creating Self-Signed Certificates (for Development)

While Zen might not have built-in cert generation, you can use OpenSSL via shell or pymport:

```
import subprocess

func generate_self_signed(domain) {
    let cmd = "openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 3
    subprocess.run(cmd, shell=true)
    print("Generated cert.pem and key.pem")
}

generate_self_signed("localhost")
```

Or with Python:

```
pymport OpenSSL.crypto

# Generate self-signed cert using pyOpenSSL
```

11.8 HTTPS Server with Certificates

```
import http

func handler(req) {
    return {
```

```
        "status": 200,
        "headers": {"Content-Type": "text/html"},
        "body": "<h1>Secure HTTPS Server</h1>"
    }
}

# Serve over HTTPS with cert and key
http.serve_tls("0.0.0.0", 443, handler, {
    "cert": "server.crt",
    "key": "server.key"
})
```

11.9 Mutual TLS (mTLS)

mTLS requires both client and server to present certificates:

```
import requests

let resp = requests.get("https://secure-api.example.com/data",
    cert=("client.crt", "client.key"),
    verify="ca.crt"
)
print(resp.text)
```

11.10 Certificate Pinning

Pin certificates to prevent MITM attacks:

```
import requests
import crypto.hash

func pinned_request(url, expected_fingerprint) {
    let resp = requests.get(url, verify=true)
    # Extract and verify certificate fingerprint
    let fingerprint = get_cert_fingerprint(resp)
    if fingerprint != expected_fingerprint {
        throw "Certificate pinning failed!"
    }
    return resp
}

# SHA256 fingerprint of expected certificate
let fp = "AA:BB:CC:..."
pinned_request("https://example.com", fp)
```

11.11 Common SSL/TLS Issues

11.11.1 Certificate Errors

Error	Cause	Solution
`CERTIFICATE_VERIFY_FAILED`	Self-signed or untrusted CA	Add CA to trust store or use `verify=false` for testing
`HOSTNAME_MISMATCH`	Cert doesn't match domain	Use correct domain or get proper cert
`CERT_HAS_EXPIRED`	Cert past expiration date	Renew certificate
`SELF_SIGNED_CERT_IN_CHAIN`	Self-signed in chain	Trust the CA or disable verification

11.11.2 Debugging SSL Issues

```
import requests

try {
    let resp = requests.get("https://problematic.site", verify=true)
} catch (e) {
    print("Error: " + e)
    if e.contains("CERTIFICATE_VERIFY_FAILED") {
        print("Certificate verification failed - check CA bundle")
    } elif e.contains("hostname") {
        print("Hostname mismatch")
    }
}
```

11.12 Let's Encrypt & Free Certificates

For production, use Let's Encrypt for free certificates. You can automate with certbot:

```
# Get certificate
certbot certonly --standalone -d example.com
```

Certificates will be in `/etc/letsencrypt/live/example.com/`

11.13 Security Best Practices

1. **Always use HTTPS in production** - No HTTP for sensitive data
2. **Verify certificates** - Never disable verification in production
3. **Use modern TLS versions** - TLS 1.2 or 1.3 only
4. **Keep certificates updated** - Monitor expiration
5. **Use strong ciphers** - Let libraries choose secure defaults
6. **Implement HSTS** - Force HTTPS with HTTP Strict Transport Security
7. **Use certificate pinning** - For high-security apps

8. **Protect private keys** - Restrict file permissions

11.14 HSTS Example

```
func handler(req) {
    return {
        "status": 200,
        "headers": {
            "Strict-Transport-Security": "max-age=31536000; includeSubDomains"
        },
        "body": "Hello"
    }
}
```

11.15 Chapter Summary

- HTTPS = HTTP + TLS encryption
- Certificates authenticate servers and enable encryption
- `requests` verifies certificates by default
- Use `crypto.x509` to inspect certificates
- Self-signed certs only for development
- Use Let's Encrypt for free production certs
- Always verify certificates in production
- Consider mTLS for mutual authentication

Next: Chapter 32 - DNS Module

12 Chapter 35: Threading Fundamentals

Zen supports multi-threaded programming, allowing your programs to execute multiple tasks concurrently. This chapter covers the fundamentals of threading in Zen.

12.1 Understanding Concurrency vs Parallelism

- **Concurrency**: Dealing with many things at once (structure)
- **Parallelism**: Doing many things at once (execution)

Threads enable both concurrent and parallel execution depending on your system's CPU cores.

12.2 Creating Threads with `spawn()`

The `spawn()` function creates a new OS thread to execute a function.

12.2.1 Basic Thread Creation

```
func worker() {
    print("Hello from worker thread!")
}

spawn(worker)
print("Hello from main thread!")
```

The order of output is non-deterministic - threads run concurrently.

12.2.2 Spawning with Anonymous Functions

```
spawn(fn() {
    print("Anonymous thread running")
    let i = 0
    while i < 3 {
        print("Thread: " + i)
        i = i + 1
    }
})

print("Main thread continues")
```

12.2.3 Passing Data to Threads

Since threads have separate execution contexts, you need to be careful about sharing data:

```
func print_message(msg, count) {
    let i = 0
    while i < count {
        print(msg + ": " + i)
        i = i + 1
    }
}

spawn(fn() { print_message("Worker1", 5) })
spawn(fn() { print_message("Worker2", 5) })
```

12.3 Thread Lifecycle

1. **Spawn**: Thread created and scheduled
2. **Running**: Executing code
3. **Finished**: Function returns, thread terminates

```
func task() {
    print("Task started")
    # Do work
    let i = 0
    while i < 1000000 {
        i = i + 1
    }
    print("Task finished")
}

spawn(task)
print("Main continues immediately")
```

12.4 Multiple Threads

```
func countdown(name, start) {
    let i = start
    while i > 0 {
        print(name + ": " + i)
        i = i - 1
    }
    print(name + ": Done!")
}

spawn(fn() { countdown("A", 3) })
spawn(fn() { countdown("B", 3) })
spawn(fn() { countdown("C", 3) })
```

12.5 Thread Safety Considerations

Threads share: - Global variables (need synchronization) - Heap objects (be careful with mutable state) - Native modules

Each thread has its own: - Stack - Local variables - Register file

12.6 Practical Example: Parallel Computation

```
let results = []
let lock = mutex.new(null)

func compute_square(n) {
    let result = n * n
    # Add to shared results with lock
    let m = mutex.lock(lock)
    results.push({n: n, square: result})
    mutex.unlock(m)
    print("Computed square of " + n)
}

# Spawn threads for parallel computation
spawn(fn() { compute_square(1) })
spawn(fn() { compute_square(2) })
spawn(fn() { compute_square(3) })
spawn(fn() { compute_square(4) })
spawn(fn() { compute_square(5) })
```

12.7 Best Practices for Threading

1. **Minimize shared mutable state** - Use channels for communication
2. **Use synchronization primitives** - Mutexes when sharing is necessary
3. **Keep threads short-lived** - Or design for long-running workers
4. **Avoid race conditions** - Always protect shared data
5. **Consider thread pool** - Don’t spawn too many threads
6. **Handle errors** - Threads can have errors too

12.8 Common Pitfalls

12.8.1 Race Condition Example

```
let counter = 0

func increment() {
    let i = 0
    while i < 1000 {
        counter = counter + 1  # NOT thread-safe!
```

```
        i = i + 1
    }
}

spawn(increment)
spawn(increment)
# counter may not be 2000 due to race condition
```

Use mutex to fix:

```
let counter = 0
let lock = mutex.new(counter)

func increment_safe() {
    let i = 0
    while i < 1000 {
        let m = mutex.lock(lock)
        let val = mutex.get(m)
        mutex.set(m, val + 1)
        counter = mutex.get(m)
        mutex.unlock(m)
        i = i + 1
    }
}
```

12.9 Chapter Summary

- Use `spawn(func)` to create threads
- Threads run concurrently and independently
- Be careful with shared mutable state
- Use channels or mutexes for synchronization
- Thread order is non-deterministic
- Each thread has its own stack and local variables

Next: Chapter 36 - Channels

13 Chapter 36: Channels

Channels provide a safe way for threads to communicate by sending and receiving values. They help avoid race conditions by synchronizing access to data.

13.1 What are Channels?

Channels are like pipes - one thread can send data, another can receive it. They handle synchronization automatically.

13.2 Creating Channels

```
let ch = channel()
```

13.3 Sending and Receiving

13.3.1 Basic Communication

```
let ch = channel()

spawn(fn() {
    ch.send("Hello from thread!")
    print("Message sent")
})

let msg = ch.recv()
print("Received: " + msg)
```

13.3.2 Bidirectional Communication

```
let ch1 = channel()
let ch2 = channel()

spawn(fn() {
    let msg = ch1.recv()
    print("Worker received: " + msg)
    ch2.send("Response from worker")
})

ch1.send("Request from main")
let response = ch2.recv()
print("Main received: " + response)
```

13.4 Blocking Behavior

`send()` and `recv()` are blocking by default: - `send()` blocks if channel is full (for buffered channels) - `recv()` blocks until a value is available

```
let ch = channel()

spawn(fn() {
    # This will block until main receives
    ch.send("data")
    print("Send completed")
})

print("Main about to receive")
let data = ch.recv()
print("Main received: " + data)
```

13.5 Producer-Consumer Pattern

This is the most common use case for channels:

```
let ch = channel()

# Producer
spawn(fn() {
    let i = 1
    while i <= 5 {
        print("Producing: " + i)
        ch.send(i)
        i = i + 1
    }
    ch.close()
    print("Producer done")
})

# Consumer
spawn(fn() {
    while true {
        let val = ch.recv()
        if val == null {
            break  # Channel closed
        }
        print("Consuming: " + val)
    }
    print("Consumer done")
})
```

13.6 Multiple Producers, Multiple Consumers

```
let ch = channel()

# Producer 1
spawn(fn() {
    let i = 0
    while i < 3 {
        ch.send("P1: " + i)
        i = i + 1
    }
})

# Producer 2
spawn(fn() {
    let i = 0
    while i < 3 {
        ch.send("P2: " + i)
        i = i + 1
    }
})

# Consumer
let i = 0
while i < 6 {
    print(ch.recv())
    i = i + 1
}
```

13.7 Closing Channels

Close a channel when no more values will be sent:

```
let ch = channel()

spawn(fn() {
    ch.send(1)
    ch.send(2)
    ch.send(3)
    ch.close()  # Signal no more data
})

# Receive until closed
while true {
    let v = ch.recv()
    if v == null {
        break
    }
```

```
    print(v)
}
```

13.8 Buffered vs Unbuffered Channels

Unbuffered (default): Sender blocks until receiver is ready

```
let ch = channel()  # Unbuffered
```

Buffered: Can hold multiple values without blocking

```
let ch = channel(10)  # Buffer size of 10
```

```
let ch = channel(3)

spawn(fn() {
    ch.send(1)
    ch.send(2)
    ch.send(3)
    print("Sent 3 items")
    ch.send(4)  # Blocks if buffer full and no receiver
    print("Sent 4th item")
})

# Consumer reads later
let i = 0
while i < 4 {
    print("Received: " + ch.recv())
    i = i + 1
}
```

13.9 Select/Multiplexing Pattern

While Zen may not have a built-in `select`, you can use patterns:

```
let ch1 = channel()
let ch2 = channel()

spawn(fn() {
    ch1.send("from ch1")
})

spawn(fn() {
    ch2.send("from ch2")
})

# Check which channel has data
# Use non-blocking patterns or just receive in order
```

13.10 Practical Example: Work Queue

```
let jobs = channel()
let results = channel()

# Worker pool
func worker(id) {
    while true {
        let job = jobs.recv()
        if job == null {
            break
        }
        print("Worker " + id + " processing job " + job)
        # Simulate work
        let result = job * 2
        results.send(result)
    }
    print("Worker " + id + " done")
}

spawn(fn() { worker(1) })
spawn(fn() { worker(2) })
spawn(fn() { worker(3) })

# Send jobs
let i = 1
while i <= 10 {
    jobs.send(i)
    i = i + 1
}
jobs.close()

# Collect results
let i = 0
while i < 10 {
    print("Result: " + results.recv())
    i = i + 1
}
```

13.11 Practical Example: Timeout Pattern

```
let ch = channel()
let timeout_ch = channel()

spawn(fn() {
    # Simulate slow operation
    # Would sleep here if sleep exists
```

```
    ch.send("slow result")
})

spawn(fn() {
    # Simulate timeout
    timeout_ch.send("timeout")
})

# Race between result and timeout
let result = ch.recv()
print(result)  # Gets whichever comes first
```

13.12 Best Practices

1. **Close channels from sender side** - Only senders should close
2. **Don't send on closed channels** - Will cause errors
3. **Use channels for communication, mutexes for shared state** - Prefer channels
4. **Buffer appropriately** - Use buffered channels to reduce blocking
5. **Handle channel closure** - Always check for null when receiving
6. **One direction per channel** - Keeps design simple

13.13 Chapter Summary

- Channels enable safe thread communication
- Use `channel()` to create, `send()` to send, `recv()` to receive
- Close with `close()`, check for null on receive
- Support buffered channels with `channel(size)`
- Perfect for producer-consumer and worker pool patterns
- Automatically handle synchronization

Next: Chapter 37 - Mutexes & Locks

14 Chapter 42: pympport - Python Interoperability

`pympport` lets you load CPython modules directly from Zen. Unlike `import` (which loads Zen bytecode), `pympport` uses an embedded CPython interpreter through the PyO3 bridge to call Python functions, access attributes, and work with Python objects as **foreign handles**.

This chapter covers: syntax, how `pympport` differs from `import`, building Zen with Python support, environment setup, dependency management with `zen py-sync` and `requirements.txt`, common patterns, limitations, and working examples (including Selenium/BeautifulSoup/MechanicalSoup and binary downloads).

14.1 42.1 Why `pympport` (and not just `import`)?

Zen keeps foreign modules on a separate keyword for correctness and tooling:

- **Parser/compiler**: `pympport` is its own statement and emits opcode `PyImport` (discriminant 81), not a flag on `Import`. Native imports stay fast and unambiguous.
- **JIT**: Functions containing `PyImport` stay **interpreted** (foreign boundaries are not lowered in v1).
- **Tooling**: A real parser scan can find all foreign dependencies for `zen py-sync` and editor integrations.
- **Determinism**: No runtime guessing of module kind via file extensions or `sys.path` probing on every resolve.

```
import fast_math        # Zen bytecode
pympport requests       # CPython via embedded interpreter
pympport selenium.webdriver as webdriver  # dotted path, aliased
```

14.2 42.2 Syntax and Binding

- `pympport dotted.path [as alias]` — one module per statement, no comma lists.
- **Binding name**: if `as alias` is given, use `alias`. Else use the **single final name** for `pympport name`. For `pympport a.b`, bind the **root package a** (Python-consistent with how imports behave).
- `a.pympport` is parsed as member access (lexer treats `pympport` after `.` as an identifier).
- The destination receives a **foreign handle** (`typeof(x) == "foreign"`).

14.3 42.3 Building Zen with Python Support

`pympport` requires an embedded CPython. Default builds are hermetic (no Python dependency). Build with the `python` feature:

```
cd /home/bard/Desktop/zen
cargo build --release --features python
```

Verify:

```
./target/release/zen run - << 'EOF'
pymport json
let data = json.loads("{\"hello\": \"world\", \"n\": 42}")
print(data["hello"])
print(data["n"])
EOF
```

If you see `Z0414` / “unknown native” or a message about `--features python`/libpython, rebuild with `--features python`.

The bridge auto-initializes on first GIL use (no cost if you don’t `pymport`).

14.4 42.4 Environment Setup (venv + PYTHONPATH)

You can point CPython to any Python environment via `PYTHONPATH`. The book ships a ready-made one in `book-v2/tests`:

```
cd book-v2/tests
./setup.sh          # builds zen --features python, creates .venv, installs requirement
```

Then, from the repository root, point Zen at that environment:

```
export PYTHONPATH="$(book-v2/tests/.venv/bin/python -c 'import sysconfig; print(sysconf
export ZEN_NO_PY_BOOTSTRAP=1
./target/release/zen run book-v2/tests/pymport/01_json_stdlib.z
```

`ZEN_NO_PY_BOOTSTRAP=1` tells Zen to use that environment as-is instead of creating `.zen_venv`.

14.4.1 Create your own venv instead

```
python3 -m venv .venv
.venv/bin/pip install beautifulsoup4 mechanicalsoup selenium lxml requests
```

Point `PYTHONPATH` at its `site-packages`. The path varies with the Python version (`python3.11`, `python3.12`, `python3.13`), so ask Python rather than hard-coding it:

```
export PYTHONPATH="$(.venv/bin/python -c 'import sysconfig; print(sysconfig.get_paths()
```

14.4.2 Check installed packages

```
book-v2/tests/.venv/bin/pip list
```

14.5 42.5 Dependency Sync with `zen py-sync` and `requirements.txt`

Zen can discover `pymport` usage and ensure a venv with dependencies.

14.5.1 Create `requirements.txt`

Put it in your project root (or nearest to the entry). Map modules to pip package names where they differ:

```
beautifulsoup4
mechanicalsoup
selenium
requests
lxml
```

> A module name is not always the pip name (e.g., `bs4` comes from `beautifulsoup4`). The scanner reports what was found; Zen does **not** guess package names.

14.5.2 Run sync

```
./target/release/zen py-sync
```

Behavior: 1. **Scan**: recursively parses entry + transitive `.z` imports to find all `pymport` statements. 2. **Plan**: determines state (`Clean`, `NoRequirements`, `MissingVenv`, `MissingPackages`). 3. **Ensure**: creates `.zen_venv` in project root, runs `pip install -r requirements.txt` in the **foreground**, writes a stamp (requirements length + hash). 4. **Point**: `zen run` prepends `.zen_venv/lib/python*/site-packages` to `PYTHONPATH`. Relative requirements paths resolve against project root.

Control flags: - `ZEN_NO_PY_BOOTSTRAP=1` — skip bootstrap entirely (for tests/hermetic runs). The runtime will still fail if packages are missing. - If `requirements.txt` is missing and `pymport` is used, Zen reports (it will not auto-install).

Example end-to-end (as tested):

```
cat > requirements.txt << 'EOF'
paramiko
EOF
./target/release/zen py-sync
./target/release/zen run my_script.z  # uses .zen_venv
```

14.6 42.6 Using Python from Zen

14.6.1 Importing modules

```
pymport json        # stdlib
pymport os          # stdlib
pymport requests    # third-party (if installed)
pymport bs4         # beautifulsoup4
pymport selenium.webdriver as webdriver
pymport urllib.request
pymport builtins
```

14.6.2 Calling functions and accessing attributes

Foreign calls are **positional-only** in v1. Use `getattr`-style access via dot notation; `call_attr` fuses resolve+dispatch for methods.

```
pymport math
print(math.sqrt(16.0))
```

```
pymport json
let d = json.loads("{\"a\": 1}")
print(d["a"])  # dict/list/indexing works on marshaled values where possible
```

14.6.3 Selenium example (positional only)

```
pymport selenium.webdriver as webdriver
pymport selenium.webdriver.chrome.options as options

let opts = options.Options()
opts.add_argument("--headless=new")
opts.add_argument("--no-sandbox")
opts.add_argument("--disable-dev-shm-usage")
let driver = webdriver.Chrome(opts)  # positional, not keyword
driver.get("https://example.com")
print(driver.title)
driver.quit()
```

14.6.4 Working with Python objects and binary data

- Python `bytes` marshal to Zen `bytes`/handles as appropriate; Python `str`/`int`/`float`/`bool`/`None`/`lis` marshal where shapes are supported.
- Tuples become lists at the boundary. Big ints beyond `i64` may become `f64` (precision caveat). Non-string dict keys are rejected.
- For binary downloads, prefer `urllib.request.urlretrieve(url, path)` or `builtins.open(path,"wb").write(py_bytes_handle)`. Native `fs.write_binary` takes a string and has string/UTF-8 constraints—treat it as text-oriented.

14.6.5 Error handling

Foreign errors surface as `NativeError`/`VmError` from the bridge with Python exception messages captured.

```
try {
    pymport nonexistent_module
} catch as e {
    print(e)
}
```

14.7 42.7 Working Examples (tested)

All scripts below live in `book-v2/tests/web-automation/`. Five smaller, hermetic pymport tests live in `book-v2/tests/pymport/`.

14.7.1 Example A: `bs4` + `requests` scrape + download

See `tests/web-automation/bs4_tiktok.z` (downloads a TikTok embed MP4 via `urllib.request.urlretrieve`, parsed with BeautifulSoup from `__FRONTITY_CONNECT_STATE__`).

14.7.2 Example B: `mechanicalsoup` stateful scrape + download

See `tests/web-automation/ms_tiktok.z`. Uses `StatefulBrowser`, accesses `browser.page`, reuses `browser.session.get()` for CDN, writes bytes via `builtins.open(...,"wb").write()`.

14.7.3 Example C: Selenium automation + download

See `tests/web-automation/selenium_tiktok.z`. Launches headless Chromium, polls for `__UNIVERSAL_DATA_FOR_REHYDRATION__`, extracts stats, fetches the embed route with plain HTTP for an unsigned MP4 URL, downloads via `urllib.request.urlretrieve`.

Run all of them (including Example D):

```
cd book-v2/tests
./run-all.sh --network
```

Or run the web examples individually:

```
cd book-v2/tests/web-automation
export PYTHONPATH="$(../.venv/bin/python -c 'import sysconfig; print(sysconfig.get_path
export ZEN_NO_PY_BOOTSTRAP=1
ZEN=../../../../target/release/zen
$ZEN run bs4_tiktok.z
$ZEN run ms_tiktok.z
timeout 300 $ZEN run selenium_tiktok.z
file out/tiktok_*.mp4
```

All produced verified `ISO Media, MP4 Base Media v1` files (~3.22 MB).

14.7.4 Example D: CDP over raw TCP/WebSocket (no Selenium)

See `tests/web-automation/cdp_tiktok.z`. Launches Chromium with `pymport subprocess`, discovers targets via HTTP `/json/list` (handle `Content-Length` and possible EAGAIN), upgrades to WebSocket, sends masked CDP frames via `sys.write_bytes(socket.fileno(sock), hex)`, parses unmasked server frames, runs `Page.navigate` + `Runtime.evaluate` to extract TikTok metadata. Cleans up with `browser.terminate()` in a `finally` block.

This demonstrates low-level socket control and binary-safe frame writing in Zen.

14.8 42.8 Limits and Non-Goals (v1)

- **No keyword arguments** in `pymport` calls.
- **No passing Zen closures into Python** (rejected by marshal).
- **No per-handle finalizers**: foreign handles remain until bridge teardown; `release` exists on the bridge but Zen GC cannot call it per-handle.
- **Tuples → lists**, `big int` → `f64` in some cases, non-string dict keys rejected.
- **Functions containing `PyImport` not JIT-lowered** (stay interpreted).
- **No `zen.toml [py-deps]`** yet (use `requirements.txt`).
- **Network installs** only via `zen py-sync`/preamble; tests avoid silent installs.
- **Handles are opaque** across isolates in general (same caveat as native handles).

14.9 42.9 Troubleshooting

Issue	Fix
`Z0414` / "unknown native" / needs `--features python`	Rebuild with `cargo build --release --features python` and ensure libpython is available.
`ModuleNotFoundError` for third-party package	Install into venv, set `PYTHONPATH`, or run `zen py-sync` with `requirements.txt`.
Selenium can't find chromedriver	Install `chromedriver` matching Chromium or put it on `PATH`. Use `--no-sandbox` in containers.
TikTok WAF blocks plain requests	Use `/embed/v2/<id>`, or Selenium/real browser. See Examples A/C.
Binary download corrupted	Don't rely on string-based `fs.write_binary`; use `urllib.request.urlretrieve` or `builtins.open("wb").write()`.
WebSocket hang in raw sockets	Read until `Content-Length` for HTTP, handle EAGAIN on keep-alive, mask client frames, parse length (126/127).
`pymport` call with kwargs fails	Use positional args only in v1.
Bootstrap skipped unexpectedly	Check `ZEN_NO_PY_BOOTSTRAP` or that `requirements.txt` exists near entry.

14.10 42.10 References

- Docs: `docs/20-pymport.md` (design/spec)
- VM bridge: `crates/zen-vm/src/pybridge.rs`, `crates/zen-vm/src/pyembed.rs`
- CLI bootstrap: `zen-cli/src/pydeps.rs`
- Tests: `crates/zen-cli/tests/pymport_e2e.rs`, `crates/zen-vm/tests/pyembed.rs`, `crates/zen-jit/tests/jit_pymport.rs`
- Scripts: `book-v2/tests/web-automation/*.z`, `book-v2/tests/pymport/*.z`

15 Chapter 44: Web Scraping & Automation

Web scraping and automation let you extract data from websites, download media, and control browsers programmatically. This chapter covers Zen's HTTP client, **pymport** (Python interoperability), browser automation with Selenium, and driving Chrome over **raw TCP/WebSocket sockets with the Chrome DevTools Protocol (CDP)**.

Every example here is runnable. The complete scripts, setup script, and test runner live in `book-v2/tests/web-automation`; the pymport fundamentals are in Chapter 42.

> **Legal and ethical note:** Only scrape sites you are permitted to scrape. Check `robots.txt`, Terms of Service, and applicable laws. The examples target publicly accessible TikTok pages for educational purposes; sites may rate-limit, require authentication, or block automated access.

15.1 Setup

From the repository root:

```
cd book-v2/tests
./setup.sh
```

`setup.sh` does three things:

1. builds Zen with the Python bridge (`cargo build --release --features python`);
2. creates `.venv/` beside the tests;
3. installs `requirements.txt`.

Then, in the shell you use for the examples:

```
cd book-v2/tests/web-automation
export PYTHONPATH="$(../.venv/bin/python -c 'import sysconfig; print(sysconfig.get_path
export ZEN_NO_PY_BOOTSTRAP=1
ZEN=../../../target/release/zen
```

Requirements:

- Zen built with `--features python`
- Python 3.9+ with `venv`
- `chromium` for Selenium and CDP
- `chromedriver` matching Chromium for Selenium
- Network access

Alternatively, let Zen manage a project environment: this directory has its own `requirements.txt`, so `zen py-sync` creates `.zen_venv/` and `zen run` uses it automatically. See Chapter 42.

> **pymport calls are positional-only.** No keyword arguments in v1. Pass `Options()` to `webdriver.Chrome(opts)` positionally, and pass `Popen` arguments positionally.

15.2 Run the whole suite

```
cd book-v2/tests
./run-all.sh              # hermetic pymport tests
./run-all.sh --network    # also runs the four examples below
```

The runner validates that each downloaded file is an MP4 (`ftyp` box) before reporting success.

15.3 1) Scrape + Download with `requests` + `bs4` (pymport)

`bs4` parses HTML. TikTok's embed page (`/embed/v2/<id>`) is accessible over plain HTTP and embeds video metadata in `__FRONTITY_CONNECT_STATE__`. The MP4 CDN URL is in `videoData.itemInfos.video.urls[0]`; `urllib.request.urlretrieve` writes it without re-encoding.

`bs4_tiktok.z`:

```
import requests
import fs
import time
pympport bs4
pympport json
pympport urllib.request
pympport socket

# requests/urllib have no default timeout; avoid hanging forever.
socket.setdefaulttimeout(45)

let UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
let VIDEO = "6718335390845095173"
let url = "https://www.tiktok.com/embed/v2/" + VIDEO

let _ = fs.mkdirs("out")
let out = "out/tiktok_bs4.mp4"

# TikTok's edge intermittently answers 503 to non-browser clients. Retry
# with backoff instead of failing the example on the first response.
let fetch = fn(u) {
    let attempt = 0
    let resp = null
    while attempt < 6 {
        resp = requests.get(u, {
            "headers": {
                "User-Agent": UA,
                "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=
                "Accept-Language": "en-US,en;q=0.9",
                "Referer": "https://www.tiktok.com/"
            },
```

```
            "timeout": 40
        })
        if resp["status_code"] == 200 {
            return resp
        }
        print("http " + str(resp["status_code"]) + " (attempt " + str(attempt + 1) + ")
        time.sleep(3 * (attempt + 1))
        attempt += 1
    }
    return resp
}

print("fetching " + url)
let resp = fetch(url)
print("status: " + str(resp["status_code"]) + " bytes: " + str(len(resp["text"])))
if resp["status_code"] != 200 {
    print("giving up: TikTok kept returning " + str(resp["status_code"]))
    exit(1)
}

let soup = bs4.BeautifulSoup(resp["text"], "html.parser")
let tag = soup.find("script", {"id": "__FRONTITY_CONNECT_STATE__"})
if tag == null {
    print("no state script")
    exit(1)
}

let data = json.loads(tag.string)
let page = data["source"]["data"]["/embed/v2/" + VIDEO]
let item = page["videoData"]["itemInfos"]
let author = page["videoData"]["authorInfos"]
let music = page["videoData"]["musicInfos"]
let video_url = item["video"]["urls"][0]

print("id: " + item["id"])
print("desc: " + item["text"])
print("author: @" + author["uniqueId"] + " (" + author["nickName"] + ")")
print("music: " + music["musicName"])
print("plays: " + str(item["playCount"]))
print("likes: " + str(item["diggCount"]))
print("comments: " + str(item["commentCount"]))
print("shares: " + str(item["shareCount"]))
print("mp4: " + video_url[0:80] + "...")

urllib.request.urlretrieve(video_url, out)
print("downloaded: " + out + " (" + str(fs.size(out)) + " bytes)")
```

Run:

```
$ZEN run bs4_tiktok.z
file out/tiktok_bs4.mp4
```

Expected: metadata for `@scout2015`, a `downloaded: out/tiktok_bs4.mp4 (3224978 bytes)` line, and `ISO Media, MP4 Base Media v1`.

15.4 2) Stateful Scraping + Download with `mechanicalsoup` (pymport)

`MechanicalSoup` wraps `requests` and BeautifulSoup in a stateful browser. `browser.open()` keeps cookies/history, `browser.page` is the current BS4 soup, and `browser.session` can be reused for the CDN fetch. Python `bytes` are written through Python's own file object because native `fs.write_binary` is string-oriented.

`ms_tiktok.z`:

```
import fs
import time
pymport mechanicalsoup
pymport json
pymport builtins
pymport socket

# requests/urllib have no default timeout; avoid hanging forever.
socket.setdefaulttimeout(45)

let UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
let VIDEO = "6718335390845095173"
let embed = "https://www.tiktok.com/embed/v2/" + VIDEO

let _ = fs.mkdirs("out")
let out = "out/tiktok_ms.mp4"

let browser = mechanicalsoup.StatefulBrowser()
# Give the session browser-like headers; MechanicalSoup otherwise sends
# python-requests' default User-Agent, which TikTok often rate-limits.
browser.session.headers.update({
    "User-Agent": UA,
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    "Accept-Language": "en-US,en;q=0.9",
    "Referer": "https://www.tiktok.com/"
})

let resp = browser.open(embed)
let attempt = 0
while resp.status_code != 200 && attempt < 6 {
    print("http " + str(resp.status_code) + " (attempt " + str(attempt + 1) + "), retry
    time.sleep(3 * (attempt + 1))
```

```
    resp = browser.open(embed)
    attempt += 1
}

print("opened: " + browser.get_url())
print("status: " + str(resp.status_code) + " bytes: " + str(len(resp.text)))
if resp.status_code != 200 {
    print("giving up: TikTok kept returning " + str(resp.status_code))
    exit(1)
}

let soup = browser.page
let tag = soup.find("script", {"id": "__FRONTITY_CONNECT_STATE__"})
if tag == null {
    print("no state script")
    exit(1)
}

let data = json.loads(tag.string)
let page = data["source"]["data"]["/embed/v2/" + VIDEO]
let item = page["videoData"]["itemInfos"]
let author = page["videoData"]["authorInfos"]
let video_url = item["video"]["urls"][0]

print("id: " + item["id"])
print("author: @" + author["uniqueId"] + " (" + author["nickName"] + ")")
print("plays: " + str(item["playCount"]) + " likes: " + str(item["diggCount"]))

# MechanicalSoup keeps a requests.Session for us: reuse it for the CDN fetch.
let bin = browser.session.get(video_url)
print("cdn status: " + str(bin.status_code) + " bytes: " + str(len(bin.content)))
if bin.status_code != 200 {
    print("giving up: CDN returned " + str(bin.status_code))
    exit(1)
}

# Python bytes -> file. Native fs.write_binary takes a string, so binary
# media is written through CPython's own file object.
let f = builtins.open(out, "wb")
f.write(bin.content)
f.close()
print("downloaded: " + out + " (" + str(fs.size(out)) + " bytes)")
```

Run:

```
$ZEN run ms_tiktok.z
```

If a site needs forms, `browser.select_form()`, fill controls, then `browser.submit_selected()`.

For real links, `browser.follow_link()` and `browser.download_link()` work as expected.

15.5 3) Browser Automation with Selenium (pymport)

Headless Chromium solves JavaScript challenges and renders pages like a user. Selenium navigates to the real TikTok video page, waits for `__UNIVERSAL_DATA_FOR_REHYDRATION__`, and extracts the full `itemStruct` (author, stats, description). The video page's `playAddr` is signed and rejected by the CDN outside the player, so the script fetches the embed route with plain HTTP for an unsigned MP4 URL.

`selenium_tiktok.z`:

```
import fs
import time
import requests
pymport selenium.webdriver as webdriver
pymport selenium.webdriver.chrome.options as options
pymport bs4
pymport json
pymport urllib.request
pymport socket

socket.setdefaulttimeout(45)

let VIDEO = "6718335390845095173"
let UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
let url = "https://www.tiktok.com/@scout2015/video/" + VIDEO

let _ = fs.mkdirs("out")
let out = "out/tiktok_selenium.mp4"

# Quit the browser and stop the chromedriver service. The embedded
# interpreter exits immediately, so this must happen before exit().
let close_browser = fn(d) {
    d.quit()
    try {
        d.service.stop()
    } catch {
        # already stopped
    }
}

let opts = options.Options()
opts.add_argument("--headless=new")
opts.add_argument("--no-sandbox")
opts.add_argument("--disable-dev-shm-usage")
opts.add_argument("--disable-gpu")
```

```
opts.add_argument("--window-size=1280,900")
opts.add_argument("--mute-audio")

print("launching chromium...")
let driver = webdriver.Chrome(opts)
driver.set_page_load_timeout(60)

print("navigating to " + url)
try {
    driver.get(url)
} catch as e {
    print("navigation warning: " + str(e))
}

# The page renders its data after the WAF/JS boot; poll for the payload.
let src = ""
let tries = 0
while tries < 15 {
    src = driver.page_source
    if src.contains("__UNIVERSAL_DATA_FOR_REHYDRATION__") {
        break
    }
    time.sleep(2)
    tries += 1
}
print("title: " + driver.title)
print("page bytes: " + str(len(src)) + " (polls: " + str(tries) + ")")

let soup = bs4.BeautifulSoup(src, "html.parser")
let tag = soup.find("script", {"id": "__UNIVERSAL_DATA_FOR_REHYDRATION__"})
if tag == null {
    print("no rehydration payload - TikTok served a challenge")
    close_browser(driver)
    exit(1)
}

let data = json.loads(tag.string)
let item = data["__DEFAULT_SCOPE__"]["webapp.video-detail"]["itemInfo"]["itemStruct"]
let stats = item["stats"]
let author = item["author"]

print("id: " + item["id"])
print("desc: " + item["desc"])
print("author: @" + author["uniqueId"] + " (" + author["nickname"] + ")")
print("plays: " + str(stats["playCount"]) + " likes: " + str(stats["diggCount"]))
print("comments: " + str(stats["commentCount"]) + " shares: " + str(stats["shareCount"]
```

```
# The video-detail page exposes a signed playAddr that the CDN rejects
# outside the player. The embed route serves an unsigned CDN URL over plain
# HTTP, so fetch that page separately after Selenium has done the hard part.
print("fetching embed route for an unsigned mp4 url...")
let embed = "https://www.tiktok.com/embed/v2/" + VIDEO
let embed_headers = {
    "User-Agent": UA,
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    "Accept-Language": "en-US,en;q=0.9",
    "Referer": "https://www.tiktok.com/"
}
let eresp = null
let eretry = 0
while eretry < 6 {
    eresp = requests.get(embed, {"headers": embed_headers, "timeout": 40})
    if eresp["status_code"] == 200 {
        break
    }
    print("embed http " + str(eresp["status_code"]) + " (attempt " + str(eretry + 1) +
    time.sleep(3 * (eretry + 1))
    eretry += 1
}
print("embed status: " + str(eresp["status_code"]))
if eresp["status_code"] != 200 {
    close_browser(driver)
    exit(1)
}
let esoup = bs4.BeautifulSoup(eresp["text"], "html.parser")
let etag = esoup.find("script", {"id": "__FRONTITY_CONNECT_STATE__"})
if etag == null {
    print("no embed state script")
    close_browser(driver)
    exit(1)
}
let edata = json.loads(etag.string)
let epage = edata["source"]["data"]["/embed/v2/" + VIDEO]
let mp4 = epage["videoData"]["itemInfos"]["video"]["urls"][0]
print("mp4: " + mp4[0:80] + "...")

urllib.request.urlretrieve(mp4, out)
print("downloaded: " + out + " (" + str(fs.size(out)) + " bytes)")

close_browser(driver)
time.sleep(1)
print("browser closed")
```

Run (Chromium startup can take 30-40s):

```
timeout 300 $ZEN run selenium_tiktok.z
```

Chromedriver must match the Chromium major version. On Debian/Ubuntu: `apt install chromium-driver`.

15.6 4) Low-Level Control: CDP over raw TCP + WebSocket sockets

This example uses no automation library. Zen launches Chromium with `--remote-debugging-port`, discovers the page target over HTTP (`/json/list`), upgrades to a WebSocket, sends masked CDP frames through `sys.write_bytes` on the socket fd, and parses unmasked server frames byte by byte.

Key protocol details:

- **HTTP**: parse headers and stop at `Content-Length`. Chrome may keep the connection open despite `Connection: close`; waiting for EOF produces a timeout (`EAGAIN`).
- **WebSocket**: client frames must be masked. Extended lengths are 126 + 16-bit or 127 + 64-bit. Server frames are not masked.
- **Binary writes**: `socket.send` takes a string and would UTF-8 re-encode masked bytes. `sys.write_bytes(fd, hex)` writes raw bytes, so the frame is built as a byte list, hex-encoded, and written to `socket.fileno(sock)`.

`cdp_tiktok.z` helpers:

```
// CDP over raw sockets: drive headless Chromium from Zen with no libraries.
import socket
import sys
import json
import time
import fs
pymport subprocess
import base64
import bytes

let PORT = 9223
let PROFILE = "out/cdp-profile"
let VIDEO = "6718335390845095173"
let VIDEO_PAGE = "https://www.tiktok.com/@scout2015/video/" + VIDEO

let HEX = "0123456789abcdef"
let to_hex = fn(bs) {
    let out = ""
    for b in bs {
        out += HEX[b >> 4] + HEX[b & 15]
    }
    return out
}
```

```
let ws_send = fn(sock, text) {
    let payload = []
    for ch in list(text) {
        for b in bytes.utf8_bytes(ch.ord()) {
            payload.push(b)
        }
    }
    let mask = [0x37, 0xA1, 0x0C, 0xF2]
    let masked = []
    let i = 0
    while i < len(payload) {
        masked.push(payload[i] ^ mask[i % 4])
        i += 1
    }
    let frame = [0x81]
    let n = len(masked)
    if n < 126 {
        frame.push(0x80 | n)
    } elif n < 65536 {
        frame.push(0x80 | 126)
        frame.push((n >> 8) & 255)
        frame.push(n & 255)
    } else {
        frame.push(0x80 | 127)
        let k = 7
        while k >= 0 {
            frame.push((n >> (8 * k)) & 255)
            k -= 1
        }
    }
    for b in mask {
        frame.push(b)
    }
    for b in masked {
        frame.push(b)
    }
    sys.write_bytes(socket.fileno(sock), to_hex(frame))
}

let recv_exact = fn(sock, n) {
    let out = []
    while len(out) < n {
        let chunk = list(socket.recv(sock, n - len(out)))
        if len(chunk) == 0 {
            throw "websocket closed"
        }
        for b in chunk {
```

```
            out.push(b)
        }
    }
    return out
}

let ws_read = fn(sock) {
    let h = recv_exact(sock, 2)
    let opcode = h[0] & 0x0F
    let masked = (h[1] & 0x80) != 0
    let n = h[1] & 0x7F
    if n == 126 {
        let e = recv_exact(sock, 2)
        n = e[0] * 256 + e[1]
    } elif n == 127 {
        let e = recv_exact(sock, 8)
        n = 0
        for b in e {
            n = n * 256 + b
        }
    }
    let mask = []
    if masked {
        mask = recv_exact(sock, 4)
    }
    let data = recv_exact(sock, n)
    if masked {
        let i = 0
        while i < len(data) {
            data[i] = data[i] ^ mask[i % 4]
            i += 1
        }
    }
    return {"opcode": opcode, "data": data}
}
```

The full script then launches Chromium with `pymport subprocess`, performs the HTTP discovery and WebSocket upgrade, and polls `Runtime.evaluate`:

```
let browser = subprocess.Popen(argv, -1, null, null, null, -3)
try {
    # GET /json/list, read until Content-Length, parse the page target
    # ... WS upgrade ...
    let nav = cdp_call(ws, 1, "Page.navigate", {"url": VIDEO_PAGE})
    let reply = cdp_call(ws, 100, "Runtime.evaluate", {
        "expression": expr,
        "returnByValue": true
    })
```

```
    # expr returns id, author, plays, likes, comments, desc
} finally {
    browser.terminate()
}
```

Run:

```
timeout 180 $ZEN run cdp_tiktok.z
```

See the complete, tested script: tests/web-automation/cdp_tiktok.z.

15.7 Tips and Gotchas

- **WAF challenges**: plain `requests` to TikTok video/profile pages may return a WAF challenge. Use `/embed/v2/<id>` or a real browser (Selenium). The embed route is reliable for the MP4 URL.
- **Binary downloads**: `fs.write_binary` expects a string and has UTF-8/string constraints. Prefer `urllib.request.urlretrieve` or `builtins.open(..., "wb").write(py_bytes)`.
- **pymport limits**: positional-only calls (no kwargs). `webdriver.Chrome(opts)` and `Popen(argv, -1, null, null, null, -3)` are positional.
- **Environment bootstrap**: use `book-v2/tests/setup.sh`, or `zen py-sync` with a local `requirements.txt`.
- **Chrome sandbox**: in containers, add `--no-sandbox` and `--disable-dev-shm-usage`.
- **Hangs**: `MechanicalSoup`, `urllib.request.urlretrieve`, and `requests` do not impose a default timeout. Set `socket.setdefaulttimeout(45)` (via `pymport socket`) or pass an explicit timeout where supported.
- **Sockets**: `socket.recv` returns `bytes`; use `list(bytes)` to index. `socket.recv_text` returns a string and is lossy for binary. For raw binary protocols, write through `sys.write_bytes(fd, hex)`.
- **JSON**: the pure-Zen `std/json.z` stringifies numbers with `str()`; nested dict access in CDP responses needs explicit keys.
- **Cleanup**: Selenium can leave `chromedriver` alive when the embedded interpreter exits; the example calls `driver.service.stop()`. For a hard-killed CDP run, remove leftovers with `pkill -x chromium`.

15.8 Verified Artifacts

Tested against real TikTok on 2026-10-23:

- `out/tiktok_bs4.mp4` — ISO Media, MP4 Base Media v1 (3,224,978 bytes)
- `out/tiktok_ms.mp4` — ISO Media, MP4 Base Media v1 (3,224,978 bytes)
- `out/tiktok_selenium.mp4` — ISO Media, MP4 Base Media v1 (3,224,978 bytes)
- CDP over raw sockets extracted metadata from `@scout2015/video/6718335390845095173`: 159600 plays, 35200 likes, 5486 comments.

15.9 Appendix: Tested Scripts

Everything in this chapter is in the repository:

- `tests/web-automation/bs4_tiktok.z` — requests + bs4 + urllib download
- `tests/web-automation/ms_tiktok.z` — MechanicalSoup + `browser.session` + `builtins` binary write
- `tests/web-automation/selenium_tiktok.z` — Selenium headless + embed route
- `tests/web-automation/cdp_tiktok.z` — raw TCP + WebSocket CDP
- `tests/web-automation/requirements.txt`
- `tests/setup.sh` — build + venv + install
- `tests/run-all.sh` — run and validate

Quick start:

```
git clone https://github.com/3CN-O4D/zen.git
cd zen/book-v2/tests
./setup.sh
./run-all.sh --network
```

See Chapter 42: pymport for setup, `zen py-sync`, and troubleshooting.

16 Chapter 45: REST API Client

Building a robust REST API client is a practical skill. This chapter walks through creating a reusable API client in Zen.

16.1 Understanding REST APIs

REST (Representational State Transfer) uses HTTP methods to perform CRUD operations: - **Create**: POST - **Read**: GET - **Update**: PUT/PATCH - **Delete**: DELETE

16.2 Basic API Client Structure

```
import requests
import json

class ApiClient {
    func init(base_url, api_key = null) {
        self.base_url = base_url
        self.api_key = api_key
        self.session = requests.Session()
        if api_key != null {
            self.session.headers["Authorization"] = "Bearer " + api_key
        }
        self.session.headers["Content-Type"] = "application/json"
        self.session.headers["User-Agent"] = "Zen-API-Client/1.0"
    }

    func get(endpoint, params = {}) {
        let url = self.base_url + endpoint
        try {
            let resp = self.session.get(url, params=params)
            return self._handle_response(resp)
        } catch (e) {
            throw "GET request failed: " + e
        }
    }

    func post(endpoint, data = {}) {
        let url = self.base_url + endpoint
        try {
            let resp = self.session.post(url, json=data)
            return self._handle_response(resp)
        } catch (e) {
            throw "POST request failed: " + e
        }
    }
```

```
    func put(endpoint, data = {}) {
        let url = self.base_url + endpoint
        try {
            let resp = self.session.put(url, json=data)
            return self._handle_response(resp)
        } catch (e) {
            throw "PUT request failed: " + e
        }
    }

    func delete(endpoint) {
        let url = self.base_url + endpoint
        try {
            let resp = self.session.delete(url)
            return self._handle_response(resp)
        } catch (e) {
            throw "DELETE request failed: " + e
        }
    }

    func _handle_response(resp) {
        if resp.status_code >= 200 && resp.status_code < 300 {
            if resp.text == "" {
                return null
            }
            try {
                return json.parse(resp.text)
            } catch {
                return resp.text
            }
        } elif resp.status_code == 401 {
            throw "Unauthorized: " + resp.text
        } elif resp.status_code == 404 {
            throw "Not Found: " + resp.text
        } elif resp.status_code >= 400 && resp.status_code < 500 {
            throw "Client Error " + resp.status_code + ": " + resp.text
        } elif resp.status_code >= 500 {
            throw "Server Error " + resp.status_code + ": " + resp.text
        }
        throw "Unexpected status: " + resp.status_code
    }
}
```

16.3 Using the API Client

```
let client = ApiClient("https://jsonplaceholder.typicode.com")

# GET all posts
let posts = client.get("/posts")
print("Got " + len(posts) + " posts")

# GET single post
let post = client.get("/posts/1")
print(post.title)

# POST create post
let new_post = client.post("/posts", {
    "title": "Hello Zen",
    "body": "This is a post from Zen",
    "userId": 1
})
print("Created post with id: " + new_post.id)

# PUT update post
let updated = client.put("/posts/1", {
    "id": 1,
    "title": "Updated Title",
    "body": "Updated body",
    "userId": 1
})
print(updated.title)

# DELETE post
client.delete("/posts/1")
print("Deleted")
```

16.4 Adding Authentication

```
# API key auth
let client = ApiClient("https://api.example.com", "your-api-key-here")

# Or OAuth token
let client2 = ApiClient("https://api.example.com")
client2.session.headers["Authorization"] = "Bearer " + oauth_token
```

16.5 Error Handling and Retries

```
import time

class RobustApiClient extends ApiClient {
    func init(base_url, api_key = null, max_retries = 3) {
```

```
        super.init(base_url, api_key)
        self.max_retries = max_retries
    }

    func get(endpoint, params = {}) {
        return self._request_with_retry(fn() {
            return super.get(endpoint, params)
        })
    }

    func _request_with_retry(func) {
        let attempts = 0
        while attempts < self.max_retries {
            try {
                return func()
            } catch (e) {
                attempts = attempts + 1
                if attempts >= self.max_retries {
                    throw e
                }
                print("Retry " + attempts + " after error: " + e)
                time.sleep(2 * attempts)  # Exponential backoff
            }
        }
    }
}
```

16.6 Pagination Handling

```
class PaginatedClient extends ApiClient {
    func get_all(endpoint, params = {}) {
        let all_results = []
        let page = 1
        let per_page = 100

        while true {
            let page_params = params.copy()
            page_params["page"] = page
            page_params["per_page"] = per_page

            let results = self.get(endpoint, page_params)
            if len(results) == 0 {
                break
            }

            for item in results {
```

```
                all_results.push(item)
            }

            if len(results) < per_page {
                break
            }

            page = page + 1
        }

        return all_results
    }
}
```

16.7 Rate Limiting

```
import time

class RateLimitedClient extends ApiClient {
    func init(base_url, api_key = null, requests_per_second = 10) {
        super.init(base_url, api_key)
        self.delay = 1.0 / requests_per_second
        self.last_request = 0
    }

    func _throttle() {
        let now = time.time()
        if now - self.last_request < self.delay {
            time.sleep(self.delay - (now - self.last_request))
        }
        self.last_request = time.time()
    }

    func get(endpoint, params = {}) {
        self._throttle()
        return super.get(endpoint, params)
    }

    func post(endpoint, data = {}) {
        self._throttle()
        return super.post(endpoint, data)
    }
}
```

16.8 Building a GitHub API Client

```
class GitHubClient extends ApiClient {
    func init(token = null) {
        super.init("https://api.github.com", token)
    }

    func get_user(username) {
        return self.get("/users/" + username)
    }

    func get_repos(username) {
        return self.get("/users/" + username + "/repos", {"sort": "stars"})
    }

    func get_repo(owner, repo) {
        return self.get("/repos/" + owner + "/" + repo)
    }

    func create_issue(owner, repo, title, body = "") {
        return self.post("/repos/" + owner + "/" + repo + "/issues", {
            "title": title,
            "body": body
        })
    }
}

# Usage
let gh = GitHubClient("your-github-token")
let user = gh.get_user("octocat")
print(user.name)
let repos = gh.get_repos("octocat")
for repo in repos[0:5] {
    print(repo.name + " - " + repo.stargazers_count)
}
```

16.9 Testing API Clients

```
# Mock responses for testing
class MockApiClient {
    func get(endpoint, params = {}) {
        if endpoint == "/users/1" {
            return {"id": 1, "name": "Test User"}
        }
        throw "Not found"
    }
}
```

```
# Use in tests
let mock = MockApiClient()
let user = mock.get("/users/1")
print(user.name)
```

16.10 Best Practices

1. **Use a base class/client** - DRY principle
2. **Handle authentication** - Store tokens securely
3. **Error handling** - Catch and provide meaningful errors
4. **Rate limiting** - Respect API limits
5. **Retries** - For transient failures
6. **Pagination** - Handle large result sets
7. **Timeouts** - Set reasonable timeouts
8. **Logging** - Log requests/responses for debugging
9. **Type safety** - Validate responses
10. **Environment variables** - Don't hardcode keys

16.11 Chapter Summary

- Build reusable API client classes
- Handle all HTTP methods
- Implement authentication, retries, pagination
- Add rate limiting and error handling
- Use sessions for efficiency
- Test with mocks
- Follow API best practices

Next: Chapter 46 - File Processing Tools

17 Chapter 46: File Processing Tools

File processing is a common task - reading logs, parsing CSV/JSON, transforming data, and batch operations. This chapter covers building practical file processing tools in Zen.

17.1 Reading and Writing Different Formats

17.1.1 CSV Processing

```
import csv

func read_csv(filename) {
    let content = fs.read(filename)
    let lines = content.split("\n")
    let headers = lines[0].split(",")
    let rows = []

    let i = 1
    while i < len(lines) {
        if lines[i].trim() == "" {
            i = i + 1
            continue
        }
        let values = lines[i].split(",")
        let row = {}
        let j = 0
        while j < len(headers) {
            row[headers[j].trim()] = values[j].trim()
            j = j + 1
        }
        rows.push(row)
        i = i + 1
    }
    return {"headers": headers, "rows": rows}
}

func write_csv(filename, headers, rows) {
    let content = headers.join(",") + "\n"
    for row in rows {
        let values = []
        for h in headers {
            values.push(row[h] ?? "")
        }
        content = content + values.join(",") + "\n"
    }
    fs.write(filename, content)
```

```
}

# Usage
let data = read_csv("users.csv")
for user in data.rows {
    print(user["name"] + " - " + user["email"])
}
```

17.1.2 JSON Processing

```
import json

func process_json_file(input_file, output_file, processor) {
    let content = fs.read(input_file)
    let data = json.parse(content)
    let result = processor(data)
    with open(output_file, "w") as f {
        f.write(json.stringify(result, indent=2))
    }
}

# Example
process_json_file("data.json", "processed.json", fn(data) {
    # Filter and transform
    let result = []
    for item in data {
        if item.active {
            result.push({
                "name": item.name.upper(),
                "email": item.email
            })
        }
    }
    return result
})
```

17.1.3 XML-like/Text Processing

```
func extract_emails_from_file(filename) {
    let content = fs.read(filename)
    let emails = re.findall(r'[\w\.-]+@[\w\.-]+\.\w+', content)
    return emails
}

func count_words(filename) {
    let content = fs.read(filename)
```

```
    let words = content.split(" ")
    return len(words)
}
```

17.2 Log File Analyzer

```
import re

class LogAnalyzer {
    func init(log_file) {
        self.log_file = log_file
    }

    func parse_logs() {
        let content = fs.read(self.log_file)
        let lines = content.split("\n")
        let entries = []

        for line in lines {
            if line.trim() == "" {
                continue
            }
            # Parse Apache log format
            let match = re.match(r'(\S+) \S+ \S+ \[([^\]]+)\] "(\S+) (\S+) ([^"]+)" (\d
            if match {
                entries.push({
                    "ip": match.group(1),
                    "timestamp": match.group(2),
                    "method": match.group(3),
                    "path": match.group(4),
                    "protocol": match.group(5),
                    "status": int(match.group(6)),
                    "size": match.group(7)
                })
            }
        }
        return entries
    }

    func count_by_status() {
        let entries = self.parse_logs()
        let counts = {}
        for entry in entries {
            let status = str(entry.status)
            if counts.contains(status) {
                counts[status] = counts[status] + 1
```

```
            } else {
                counts[status] = 1
            }
        }
        return counts
    }

    func top_ips(limit = 10) {
        let entries = self.parse_logs()
        let ip_counts = {}
        for entry in entries {
            if ip_counts.contains(entry.ip) {
                ip_counts[entry.ip] = ip_counts[entry.ip] + 1
            } else {
                ip_counts[entry.ip] = 1
            }
        }
        # Sort and return top
        let items = ip_counts.items()
        # Simple sort would go here
        return items
    }
}

let analyzer = LogAnalyzer("access.log")
print(analyzer.count_by_status())
```

17.3 Batch File Processor

```
import path

func process_directory(dir, pattern, processor) {
    let files = fs.listdir(dir)
    for filename in files {
        if filename.matches(pattern) {
            let full_path = path.join(dir, filename)
            print("Processing: " + full_path)
            processor(full_path)
        }
    }
}

# Rename files
process_directory("./images", "*.jpg", fn(filepath) {
    let new_name = filepath.replace(".jpg", ".jpeg")
    fs.rename(filepath, new_name)
```

```
    print("Renamed to " + new_name)
})

# Convert files
process_directory("./data", "*.txt", fn(filepath) {
    let content = fs.read(filepath)
    let upper = content.upper()
    fs.write(filepath + ".upper", upper)
})
```

17.4 Data Transformation Pipeline

```
class Pipeline {
    func init(data) {
        self.data = data
    }

    func filter(predicate) {
        let result = []
        for item in self.data {
            if predicate(item) {
                result.push(item)
            }
        }
        self.data = result
        return self
    }

    func map(transform) {
        let result = []
        for item in self.data {
            result.push(transform(item))
        }
        self.data = result
        return self
    }

    func sort(key_func) {
        # Simple bubble sort
        let arr = self.data
        let n = len(arr)
        let i = 0
        while i < n {
            let j = 0
            while j < n - i - 1 {
                if key_func(arr[j]) > key_func(arr[j+1]) {
```

```
                    let tmp = arr[j]
                    arr[j] = arr[j+1]
                    arr[j+1] = tmp
                }
                j = j + 1
            }
            i = i + 1
        }
        self.data = arr
        return self
    }

    func take(n) {
        self.data = self.data[0:n]
        return self
    }

    func collect() {
        return self.data
    }
}

# Usage
let users = read_csv("users.csv").rows
let result = Pipeline(users)
    .filter(fn(u) { return int(u["age"]) > 25 })
    .map(fn(u) { return {"name": u["name"], "email": u["email"]} })
    .sort(fn(u) { return u["name"] })
    .take(10)
    .collect()

print(json.stringify(result))
```

17.5 File Watcher

```
func watch_directory(dir, callback) {
    let last_state = {}
    # Initialize
    let files = fs.listdir(dir)
    for f in files {
        last_state[f] = fs.mtime(path.join(dir, f))
    }

    while true {
        let current = {}
        let files = fs.listdir(dir)
```

```
        for f in files {
            let p = path.join(dir, f)
            current[f] = fs.mtime(p)
        }

        # Check for new/modified files
        for f in current.keys() {
            if !last_state.contains(f) {
                callback(f, "created")
            } elif current[f] != last_state[f] {
                callback(f, "modified")
            }
        }

        # Check for deleted files
        for f in last_state.keys() {
            if !current.contains(f) {
                callback(f, "deleted")
            }
        }

        last_state = current
        time.sleep(1)
    }
}

watch_directory("./watch", fn(filename, event) {
    print(event + ": " + filename)
})
```

17.6 Text File Merger and Splitter

```
func merge_files(file_list, output_file) {
    let content = ""
    for f in file_list {
        content = content + fs.read(f) + "\n"
    }
    fs.write(output_file, content)
    print("Merged " + len(file_list) + " files")
}

func split_file(filename, lines_per_file) {
    let content = fs.read(filename)
    let lines = content.split("\n")
    let i = 0
    let part = 0
```

```
    while i < len(lines) {
        let chunk = lines[i : i + lines_per_file]
        let out_name = filename + ".part" + part
        fs.write(out_name, chunk.join("\n"))
        print("Created " + out_name)
        i = i + lines_per_file
        part = part + 1
    }
}
```

17.7 Data Validation

```
func validate_csv_row(row, rules) {
    for field in rules.keys() {
        if rules[field].required && (!row.contains(field) || row[field] == "") {
            throw "Missing required field: " + field
        }
        if row.contains(field) && rules[field].type == "email" {
            if !row[field].contains("@") {
                throw "Invalid email in " + field
            }
        }
        if row.contains(field) && rules[field].type == "number" {
            try {
                int(row[field])
            } catch {
                throw "Invalid number in " + field
            }
        }
    }
    return true
}

let rules = {
    "email": {"required": true, "type": "email"},
    "age": {"required": true, "type": "number"},
    "name": {"required": true}
}

let data = read_csv("users.csv")
for row in data.rows {
    try {
        validate_csv_row(row, rules)
        print("Valid: " + row["name"])
    } catch (e) {
        print("Invalid: " + e)
```

```
    }
}
```

17.8 Chapter Summary

- Process CSV, JSON, and text files
- Build log analyzers and data pipelines
- Create batch processors for directories
- Watch directories for changes
- Merge, split, and transform files
- Validate data during processing
- Use functional patterns with pipelines

Next: Chapter 47 - Network Security Tools

18 Chapter 47: Network Security Tools

Note: This chapter is for educational and ethical purposes only. Always get proper authorization before testing network security.

18.1 Port Scanner

```
import socket
import time

func scan_port(host, port, timeout=1) {
    try {
        let s = socket.create(socket.AF_INET, socket.SOCK_STREAM)
        socket.settimeout(s, timeout)
        socket.connect(s, host, port)
        socket.close(s)
        return true
    } catch {
        return false
    }
}

func scan_ports(host, start_port, end_port) {
    print("Scanning " + host + " from port " + start_port + " to " + end_port)
    let open_ports = []
    let port = start_port
    while port <= end_port {
        if scan_port(host, port) {
            print("Port " + port + " is OPEN")
            open_ports.push(port)
        }
        port = port + 1
    }
    return open_ports
}

# Usage (only scan hosts you own or have permission to scan)
# let open = scan_ports("127.0.0.1", 1, 1024)
```

18.2 Subdomain Finder

```
import dns
import requests

func find_subdomains(domain, wordlist) {
    let found = []
```

```
    for sub in wordlist {
        let hostname = sub + "." + domain
        try {
            let ips = dns.resolve(hostname)
            print("Found: " + hostname + " -> " + ips[0])
            found.push({"subdomain": hostname, "ips": ips})
        } catch {
            # Not found
        }
    }
    return found
}

let wordlist = ["www", "mail", "ftp", "api", "admin", "test", "dev"]
# find_subdomains("example.com", wordlist)
```

18.3 HTTP Header Checker

```
import requests

func check_headers(url) {
    try {
        let resp = requests.get(url)
        print("Security Headers for " + url)
        print("=" * 40)

        let security_headers = [
            "Strict-Transport-Security",
            "Content-Security-Policy",
            "X-Frame-Options",
            "X-Content-Type-Options",
            "X-XSS-Protection",
            "Referrer-Policy",
            "Permissions-Policy"
        ]

        for h in security_headers {
            if resp.headers.contains(h) {
                print("✓ " + h + ": " + resp.headers[h])
            } else {
                print("✗ " + h + ": MISSING")
            }
        }
    } catch (e) {
        print("Error: " + e)
    }
```

```
}

check_headers("https://example.com")
```

18.4 Directory Brute Forcer

```
import requests

func brute_directories(url, wordlist) {
    let found = []
    for word in wordlist {
        let test_url = url + "/" + word
        try {
            let resp = requests.get(test_url)
            if resp.status_code == 200 || resp.status_code == 301 || resp.status_code =
                print("Found: " + test_url + " [" + resp.status_code + "]")
                found.push(test_url)
            }
        } catch {}
    }
    return found
}

let dirs = ["admin", "login", "config", "backup", "uploads"]
# brute_directories("https://example.com", dirs)
```

18.5 SSL Certificate Checker

```
import crypto.x509
import socket
import ssl

func check_ssl_cert(host, port=443) {
    try {
        # Get certificate (simplified)
        let cert_pem = get_server_cert(host, port)
        let cert = crypto.x509.load_cert(cert_pem)

        print("Certificate for " + host)
        print("Subject:", cert.subject)
        print("Issuer:", cert.issuer)
        print("Valid until:", cert.not_after)

        # Check expiration
        let days_left = (cert.not_after_timestamp - time.time()) / 86400
        if days_left < 30 {
```

```
            print("WARNING: Expires in " + int(days_left) + " days!")
        }
    } catch (e) {
        print("Error: " + e)
    }
}
```

18.6 Ethical Guidelines

1. **Only test with permission** - Written authorization required
2. **Stay in scope** - Only test authorized targets
3. **Report responsibly** - Report vulnerabilities properly
4. **No malicious use** - These tools for defense/testing only
5. **Follow laws** - Computer crime laws apply
6. **Educational purpose** - Learn to defend, not attack

18.7 Chapter Summary

- Port scanning for network reconnaissance
- Subdomain enumeration
- HTTP security header analysis
- Directory enumeration
- SSL certificate inspection
- Always practice ethical security

Next: Chapter 48 - Capstone Projects

19 Chapter 48: Capstone Projects

Put everything together with these complete, real-world projects.

19.1 Project 1: Full-Stack Blog API

Build a complete blog API with authentication, posts, comments.

```
import http
import json
import crypto.jwt
import crypto.rand

let JWT_SECRET = "change-in-production"
let posts = []
let users = []

func hash_password(pw) {
    # Simplified
    return crypto.hash.sha256(pw).hexdigest()
}

func auth_middleware(req, handler) {
    if req.path.starts_with("/api/protected") {
        let auth = req.headers.get("Authorization", "")
        if !auth.starts_with("Bearer ") {
            return {"status": 401, "body": json.stringify({"error": "Unauthorized"})}
        }
        try {
            let token = auth.split(" ")[1]
            let payload = crypto.jwt.decode(token, JWT_SECRET)
            req.user = payload
        } catch {
            return {"status": 401, "body": json.stringify({"error": "Invalid token"})}
        }
    }
    return handler(req)
}

func router(req) {
    # Register
    if req.path == "/api/register" && req.method == "POST" {
        let body = json.parse(req.body)
        let user = {
            "id": len(users) + 1,
            "username": body.username,
            "password": hash_password(body.password)
        }
```

```
        users.push(user)
        return {"status": 201, "body": json.stringify({"id": user.id})}
    }

    # Login
    if req.path == "/api/login" && req.method == "POST" {
        let body = json.parse(req.body)
        for user in users {
            if user.username == body.username && user.password == hash_password(body.pa
                let token = crypto.jwt.encode({"user_id": user.id}, JWT_SECRET)
                return {"status": 200, "body": json.stringify({"token": token})}
            }
        }
        return {"status": 401, "body": json.stringify({"error": "Invalid credentials"})
    }

    # Get posts
    if req.path == "/api/posts" && req.method == "GET" {
        return {"status": 200, "body": json.stringify(posts)}
    }

    # Create post (protected)
    if req.path == "/api/protected/posts" && req.method == "POST" {
        let body = json.parse(req.body)
        let post = {
            "id": len(posts) + 1,
            "title": body.title,
            "content": body.content,
            "author_id": req.user.user_id
        }
        posts.push(post)
        return {"status": 201, "body": json.stringify(post)}
    }

    return {"status": 404, "body": json.stringify({"error": "Not found"})}
}

func main() {
    print("Blog API running on http://localhost:8080")
    http.serve("0.0.0.0", 8080, fn(req) { return auth_middleware(req, router) })
}

main()
```

19.2 Project 2: Real-Time Chat Application

```
import socket
import json

let clients = []
let client_counter = 0

func broadcast(message, sender_id) {
    for client in clients {
        if client.id != sender_id {
            try {
                socket.send(client.sock, json.stringify(message) + "\n")
            } catch {}
        }
    }
}

func handle_client(client) {
    while true {
        try {
            let data = socket.recv(client.sock, 1024)
            if len(data) == 0 {
                break
            }
            let msg = json.parse(data.trim())
            broadcast({
                "type": "message",
                "from": client.name,
                "text": msg.text
            }, client.id)
        } catch {
            break
        }
    }
    # Remove client
    socket.close(client.sock)
    print(client.name + " disconnected")
}

func main() {
    let server = socket.create(socket.AF_INET, socket.SOCK_STREAM)
    socket.setsockopt(server, socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    socket.bind(server, "0.0.0.0", 9000)
    socket.listen(server, 100)

    print("Chat server on port 9000")
```

```
    while true {
        let sock = socket.accept(server)
        client_counter = client_counter + 1
        let client = {"id": client_counter, "sock": sock, "name": "User" + client_count
        clients.push(client)
        print("New connection: " + client.name)
        spawn(fn() { handle_client(client) })
    }
}

main()
```

19.3 Project 3: Web Scraper with Data Export

```
import requests
import json
import re
import time

class Scraper {
    func init() {
        self.results = []
    }

    func scrape_quotes() {
        let page = 1
        while page <= 3 {
            let url = "https://quotes.toscrape.com/page/" + page + "/"
            try {
                let resp = requests.get(url)
                # Extract quotes (simplified pattern)
                let quotes = re.findall(r'<span class="text" itemprop="text">([^<]+)</s
                let authors = re.findall(r'<small class="author" itemprop="author">([^<

                let i = 0
                while i < len(quotes) {
                    self.results.push({
                        "quote": quotes[i],
                        "author": authors[i]
                    })
                    i = i + 1
                }
                print("Scraped page " + page)
                time.sleep(1)
            } catch (e) {
                print("Error: " + e)
```

```
            }
            page = page + 1
        }
    }

    func export_json(filename) {
        with open(filename, "w") as f {
            f.write(json.stringify(self.results, indent=2))
        }
        print("Exported to " + filename)
    }

    func export_csv(filename) {
        let content = "quote,author\n"
        for r in self.results {
            content = content + '"' + r.quote.replace('"', '""') + '","' + r.author.rep
        }
        fs.write(filename, content)
        print("Exported to " + filename)
    }
}

let scraper = Scraper()
scraper.scrape_quotes()
scraper.export_json("quotes.json")
scraper.export_csv("quotes.csv")
```

19.4 Project 4: Task Scheduler

```
import time
import json

class TaskScheduler {
    func init() {
        self.tasks = []
    }

    func add_task(name, func, interval_seconds) {
        self.tasks.push({
            "name": name,
            "func": func,
            "interval": interval_seconds,
            "last_run": 0
        })
        print("Added task: " + name)
    }
```

```
    func run() {
        print("Task scheduler started")
        while true {
            let now = time.time()
            for task in self.tasks {
                if now - task.last_run >= task.interval {
                    print("Running task: " + task.name)
                    try {
                        task.func()
                    } catch (e) {
                        print("Task error: " + e)
                    }
                    task.last_run = now
                }
            }
            time.sleep(1)
        }
    }
}

let scheduler = TaskScheduler()

scheduler.add_task("heartbeat", fn() {
    print("Heartbeat: " + time.time())
}, 10)

scheduler.add_task("cleanup", fn() {
    print("Cleaning up...")
}, 60)

scheduler.run()
```

19.5 Final Words

Congratulations on completing the comprehensive Zen for Dummies guide! You've learned:

- **Fundamentals**: Data types, functions, lambdas, closures
- **Core Concepts**: Modules, error handling, classes, objects
- **Networking**: Sockets, HTTP, HTTPS, SSL/TLS, DNS
- **Concurrency**: Threads, channels, mutexes
- **Cryptography**: Hashing, encryption, JWT, certificates
- **Practical Skills**: Web scraping, API clients, file processing
- **Projects**: Real-world applications

19.6 Next Steps

1. **Practice** - Build your own projects
2. **Read the Source** - Explore Zen's stdlib and codebase 3 **Document** - Write comments and documentation
3. **Share** - Contribute to the Zen community
4. **Keep Learning** - Experiment with new features

Remember: The best way to learn is by doing. Start with small projects and gradually build up to more complex ones.

Happy coding with Zen!

End of Zen for Dummies: Complete Edition